Group Assignment

Java Programming

(AAPP004-4-2)

Lecturer: Mr Kau Guan Kiat

UCDF2204ICT (DI)

Group Members

1. Beh Swee Shen TP068561
2. Lim Beng Rhui TP068495

Table of Contents

Introduction

As one of Malaysia's top private universities, the Asia Pacific University of Technology and Innovation (APU) has offered students a place to stay, especially those from another city or country, by providing multiple accommodations. Not only does the accommodation of the university offer several facilities like a gym room and convenience stores, but the university has also strived to provide the best services for its students by providing services, such as appliance services, where students can send broken appliances to the technician-in-charge to repair, all with a low service fee. To better serve the accommodation tenants, the university has decided to develop a system known as the **APU Hostel Home Appliances Service Centre (AHHASC)** that allows managers and technicians of the service centre to handle appointments for any services swiftly. In this project, our team has developed a system for the service centre using the Java programming language, alongside implementing object-oriented concepts to manage the complexities involved in developing the project. It is hoped that with the developed system, our team will be able to assist the personnel in the service centre in smoothening their working process and providing higher-level services to APU students.

Assumptions

Before working on the coding development of the AHHASC system, our team has outlined a list of assumptions that provide an overall direction for the system development.

General Assumption

- The AHHASC system is catered for three users: managers, technicians, and customers.
- All system data is stored in text files.
- The latest version of the Java Development Kit (JDK) has to be installed so that the user can run the system developed using the Java programming language.
- The system developed will be a graphical user interface (GUI).
- Customers are assumed to be the students staying at APU's accommodation, where each customer will have their unique identification number, known as the TP number, to access their previous and upcoming appointments on the system.
- Customers will have to rely on the system's managers to book appointments. This can be done by emailing the managers or calling the service centre.
- Appointments are only available from 9:00 a.m. till 6:00 p.m..

Manager

- Managers must enter credentials, including their email and password, to log in to the system.
- Managers will enjoy the following features via the system:
    - Create accounts for all users: manager, technician, and customer accounts.
    - Modify the account information for all users, such as managers, technicians, and customers, which involves changing the user type for managers and technicians.
    - Book appointments for customers by selecting a suitable technician and time slot based on the negotiation between the customer and the manager
    - View the summary of customer feedback, including the overall system ratings, technician ratings, and individual customer comments.
    - Update the list of items available to be serviced together with their price.
- The AHHASC system only provides three positions for managers, namely the chief executive officer (CEO), chief financial officer (CFO), and warden.
- If the manager were to change the user type of the technician account to the manager, all the technician's previous appointment details would be removed from the system.

Technician

- Technician shall work under the booking assignments appointed by the manager.
- Technician must enter their credentials to gain access to the system.
- Technician will be able to enjoy the following features from the system:
    - Modify the technician's individual account information and reset password
    - View the appointments of the current day
    - View all previous appointments that has been assigned by the manager to the customers
    - Record customer payments that are made face-to-face during the service by changing the payment status on the system
    - View own ratings and individual customer feedbacks limited to the technician
- For technicians, the positions available are electrician, computer technician and maintenance technician.

Customers

- Customer will be able to interact with the system to view previous and upcoming appointments.
- Customer can interact with the system to provide feedback for their previous appointments.
- Once the customer provides a feedback, the customer will not be able to view or modify the feedback anymore.

Flowchart

Our team has also developed flowcharts to depict the user interaction with the system.

Overall Process

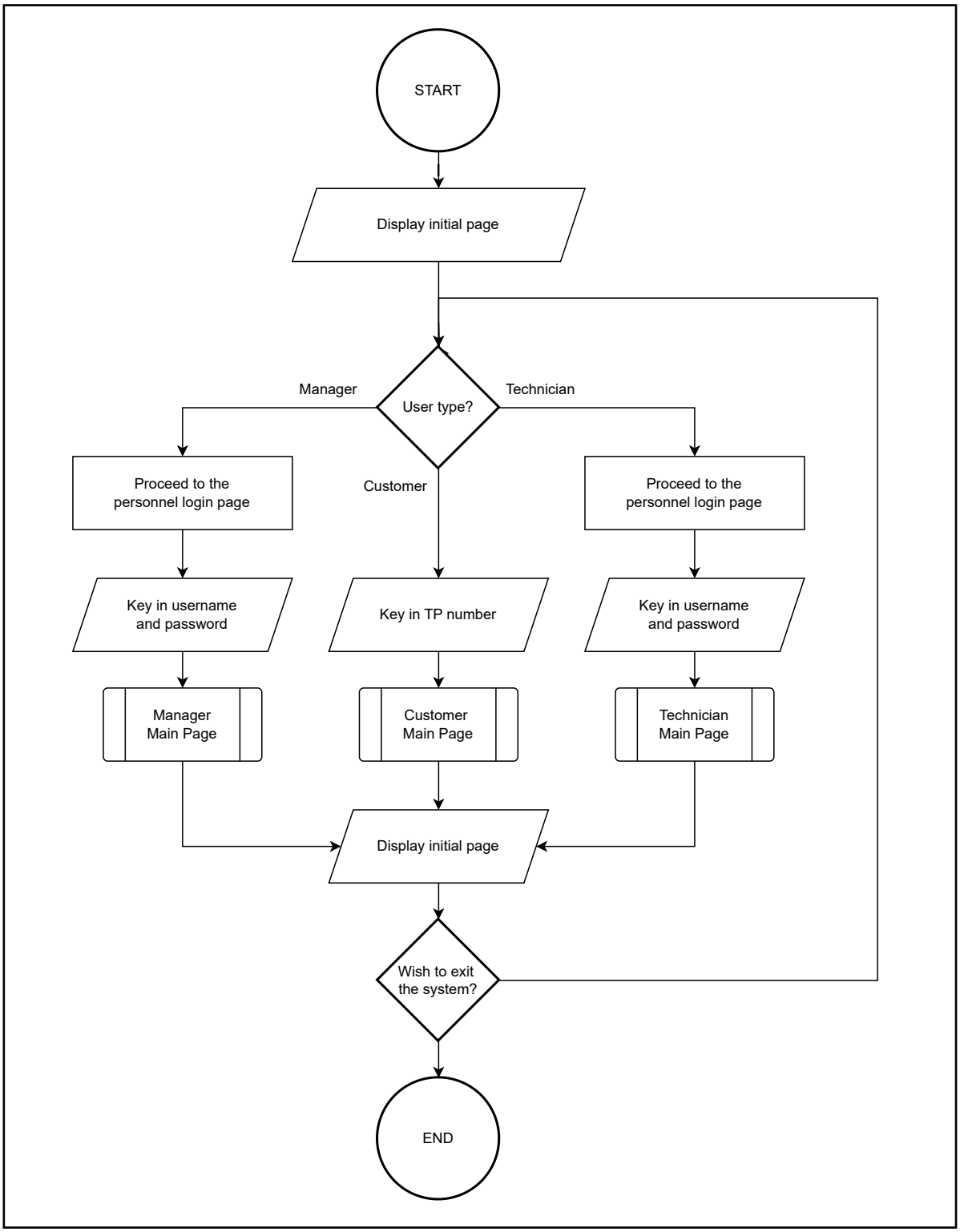

***Figure 1*: Flowchart for Overall Process.**

Manager Main Page

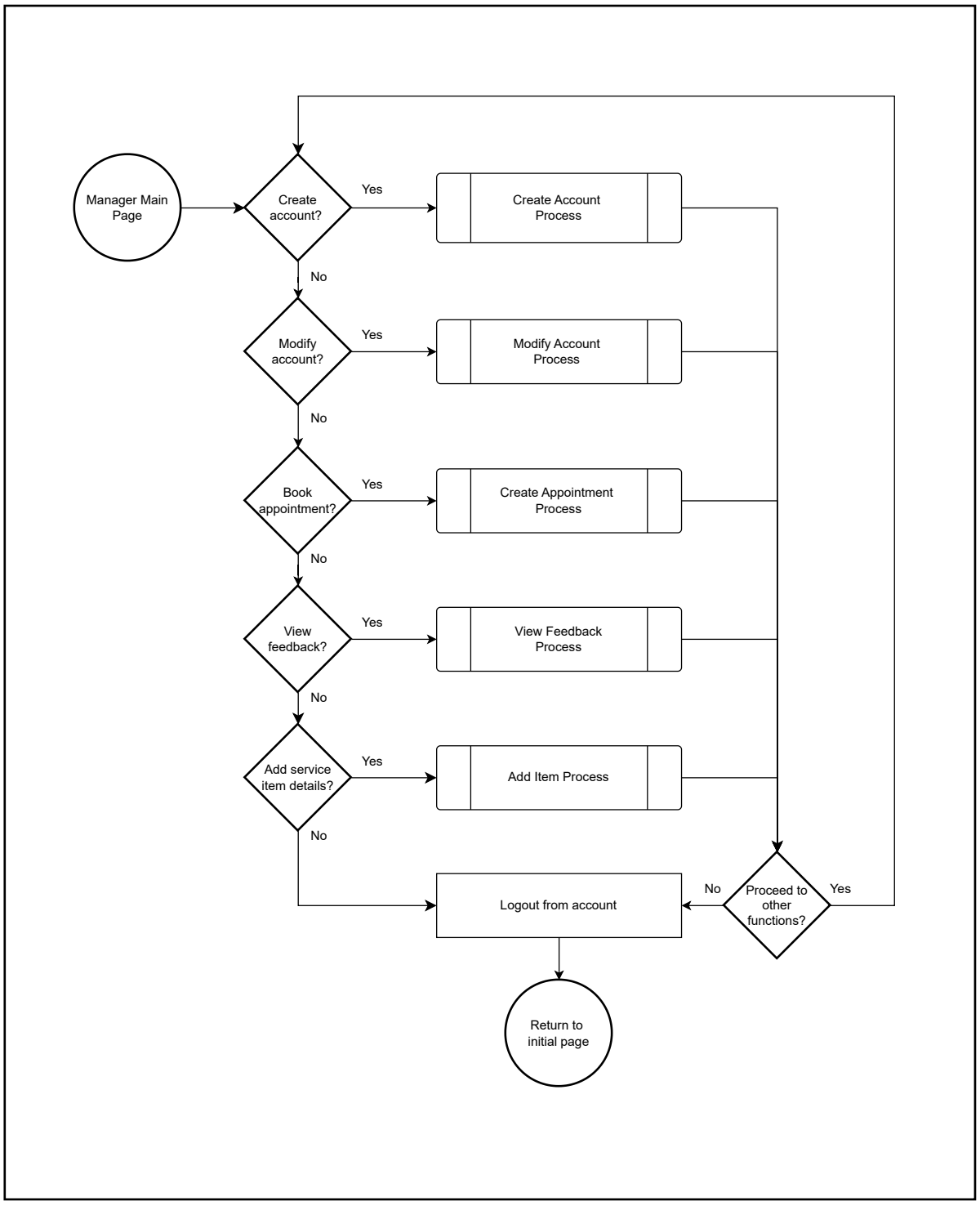

***Figure 2*: Flowchart for Manager Main Page.**

Manager Create Account

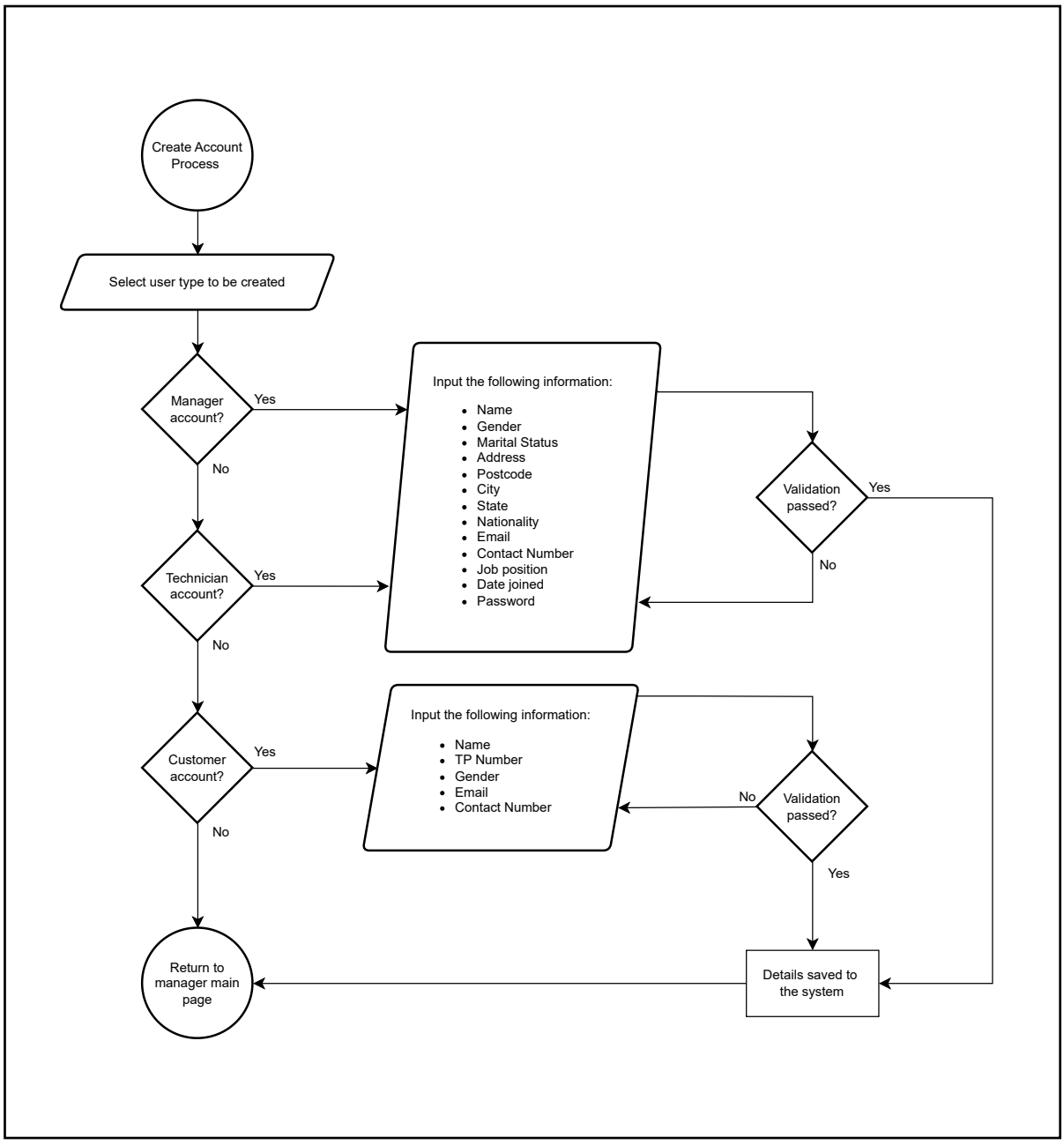

***Figure 3*: Flowchart for Manager Create Account.**

Manager Modify Account

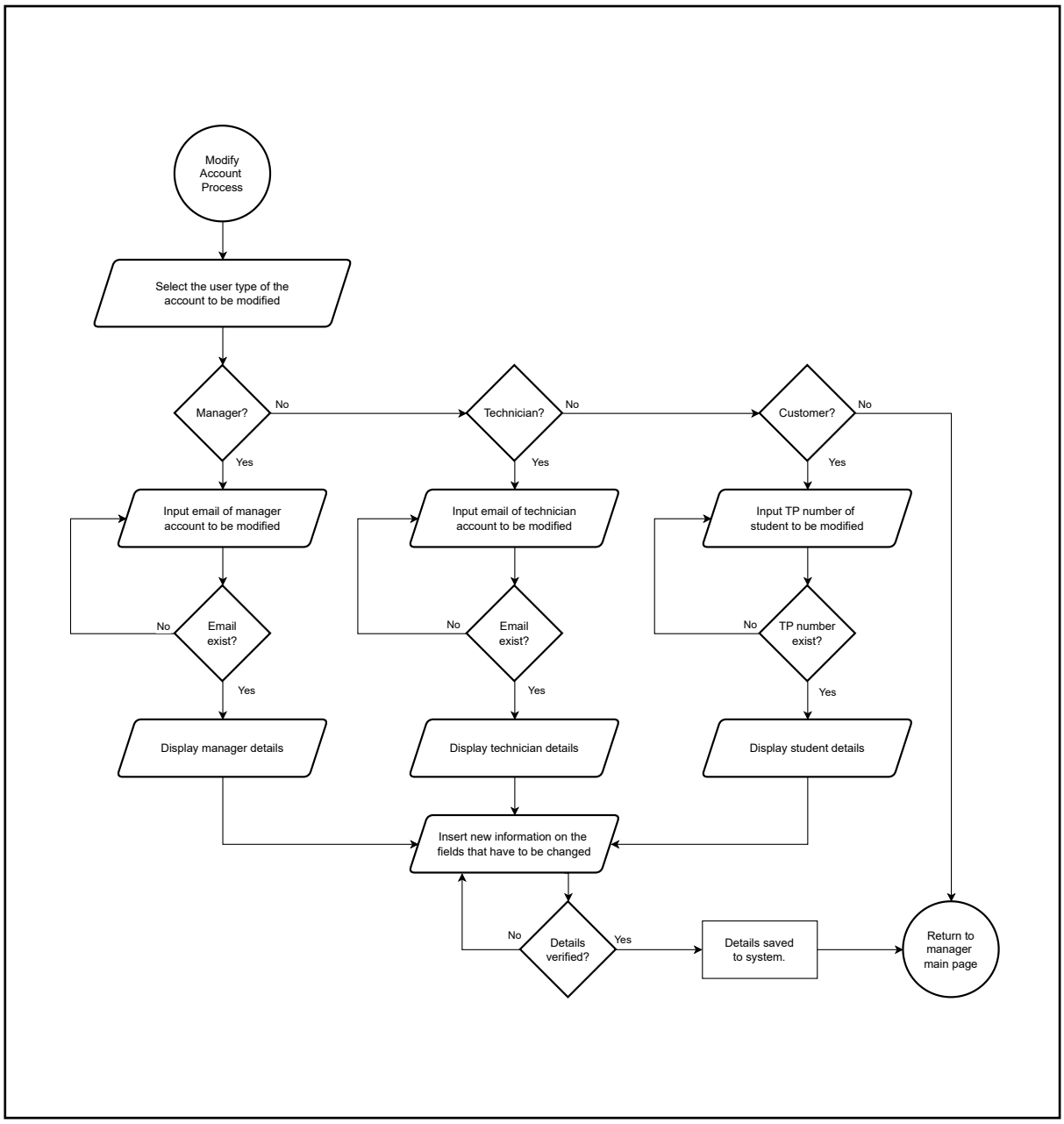

***Figure 4*: Flowchart for Manager Modify Account.**

Manager Create Appointment

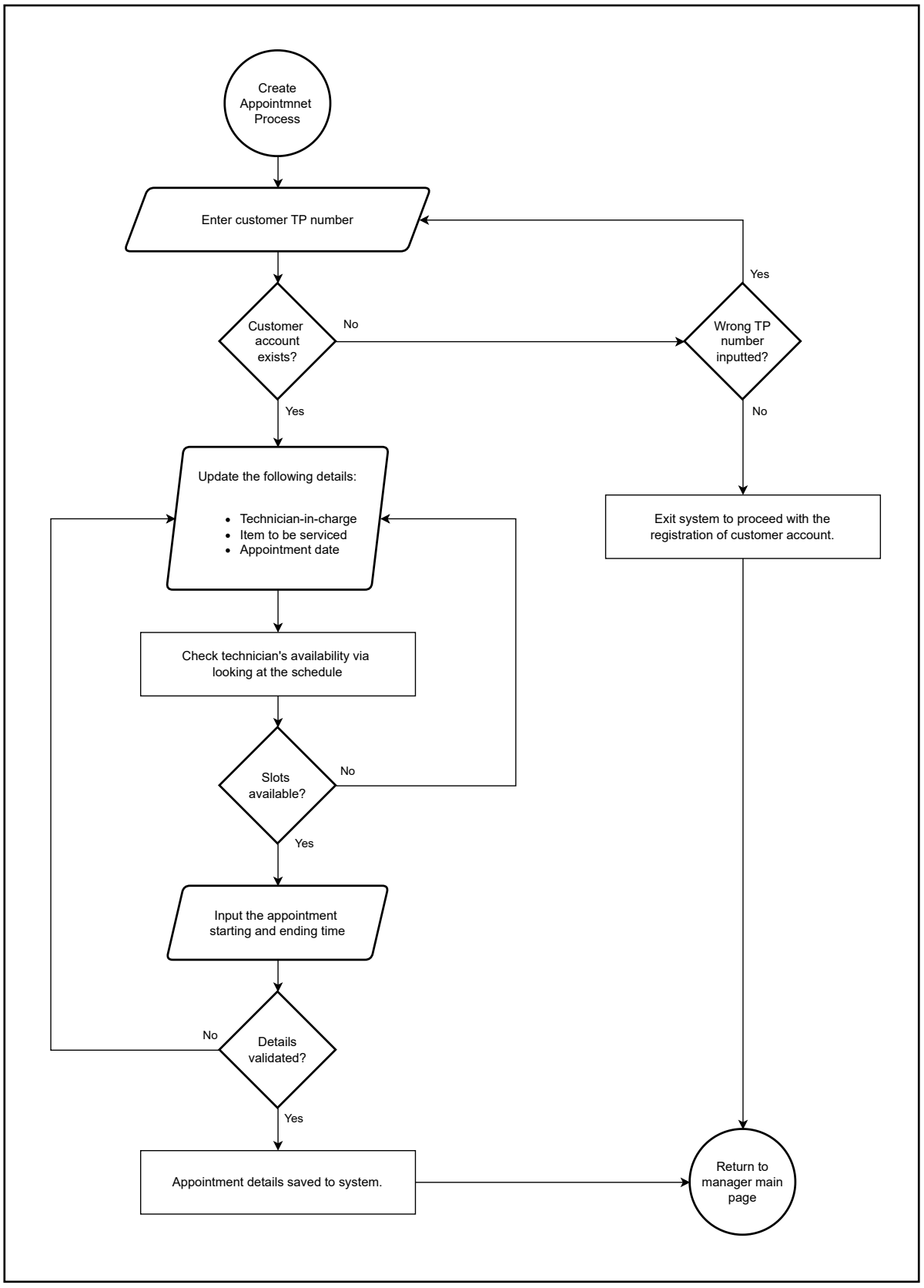

Figure 5: **Flowchart for Manager Create Appointment.**

Manager View Feedback

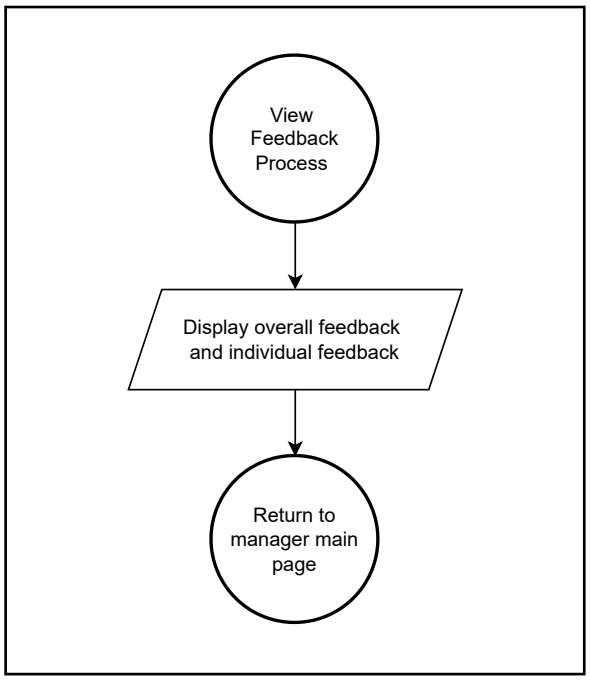

***Figure 6*: Flowchart for Manager View Feedback.**

Manager Add Items

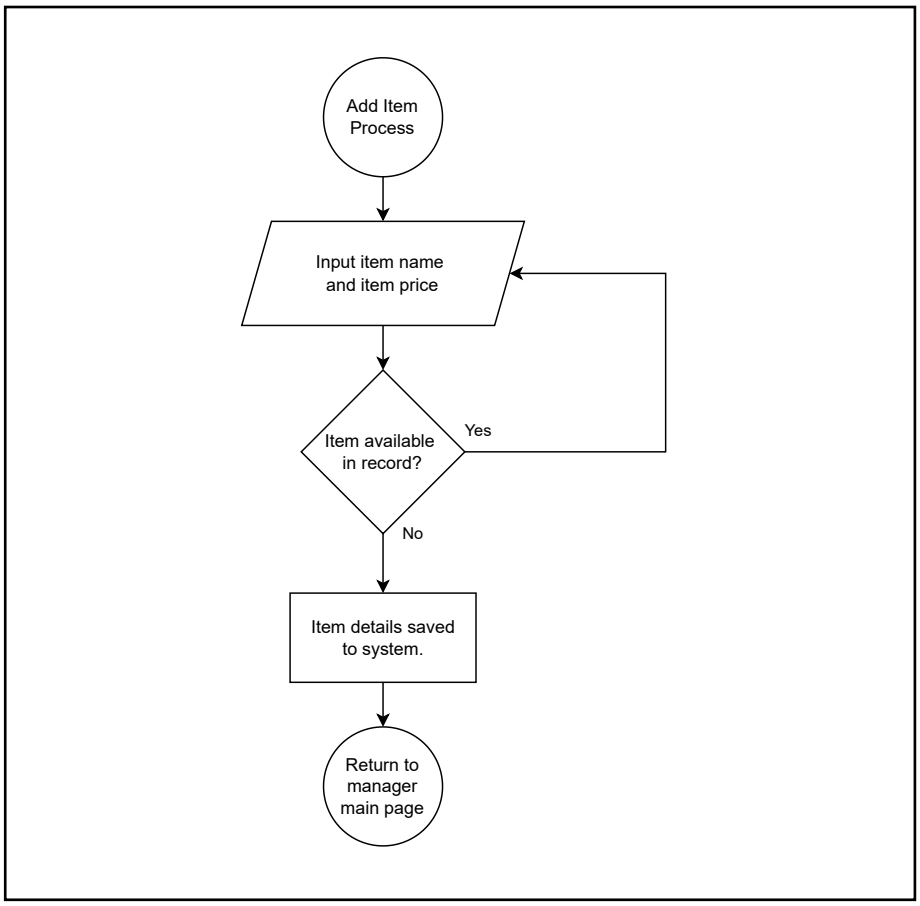

***Figure 7*: Flowchart for Manager Add Items.**

Technician Main Page

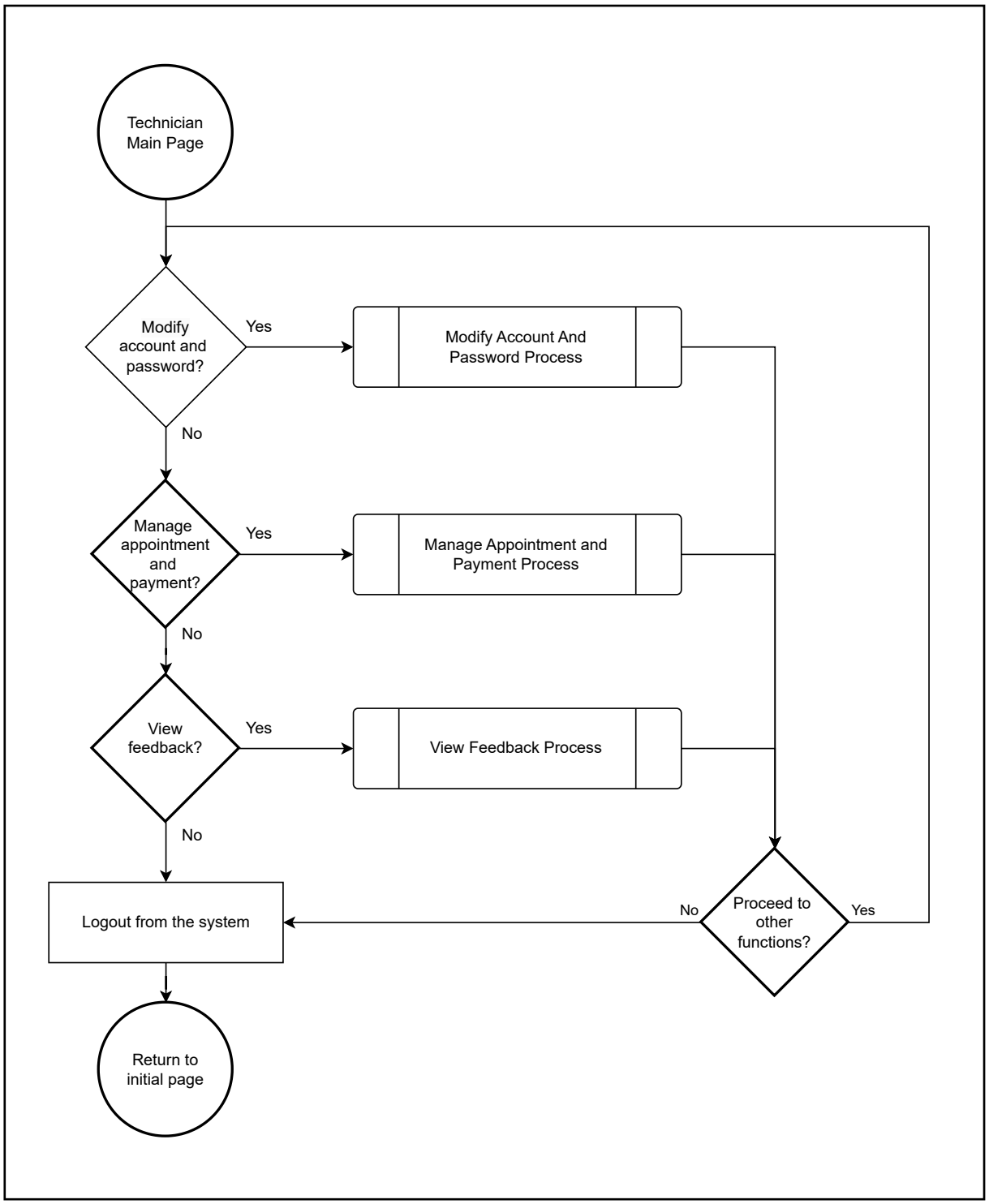

Figure 8: **Flowchart for Technician Main Page.**

Technician Modify Account and Password

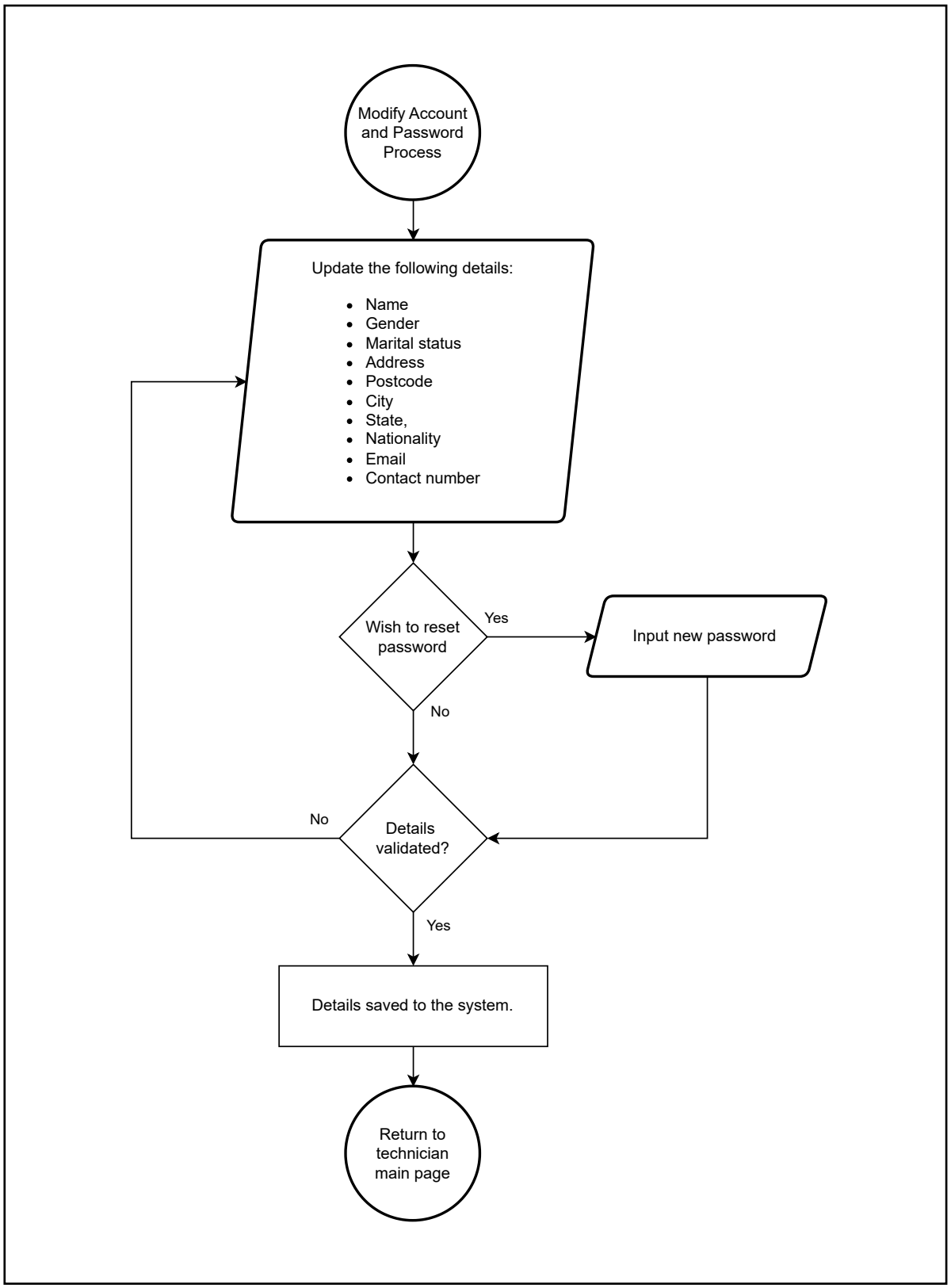

***Figure 9*: Technician Modify Account and Password.**

Technician Manage Payment and Appointment

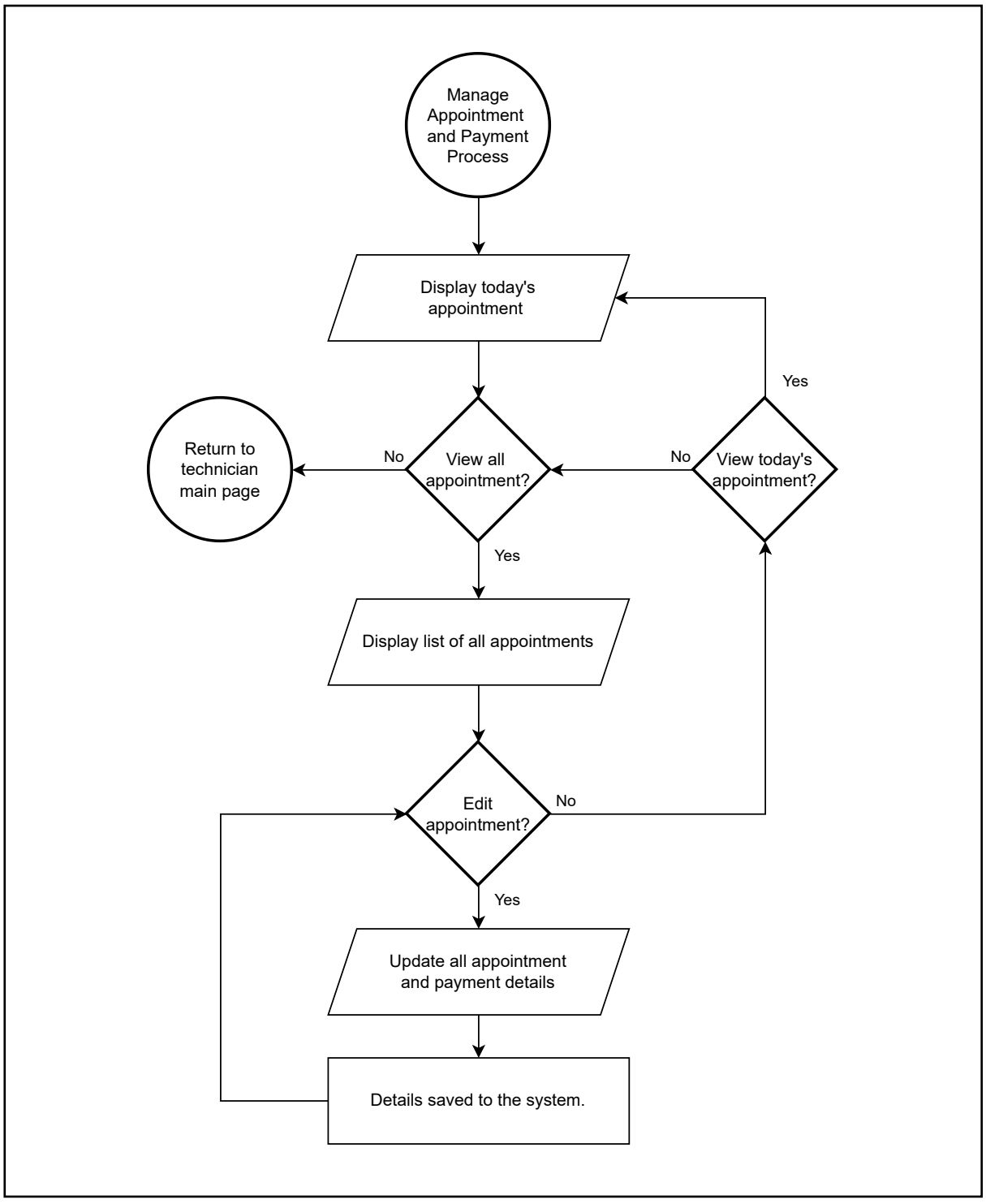

***Figure 10*: Technician Manage Payment and Appointment.**

Technician View Feedback

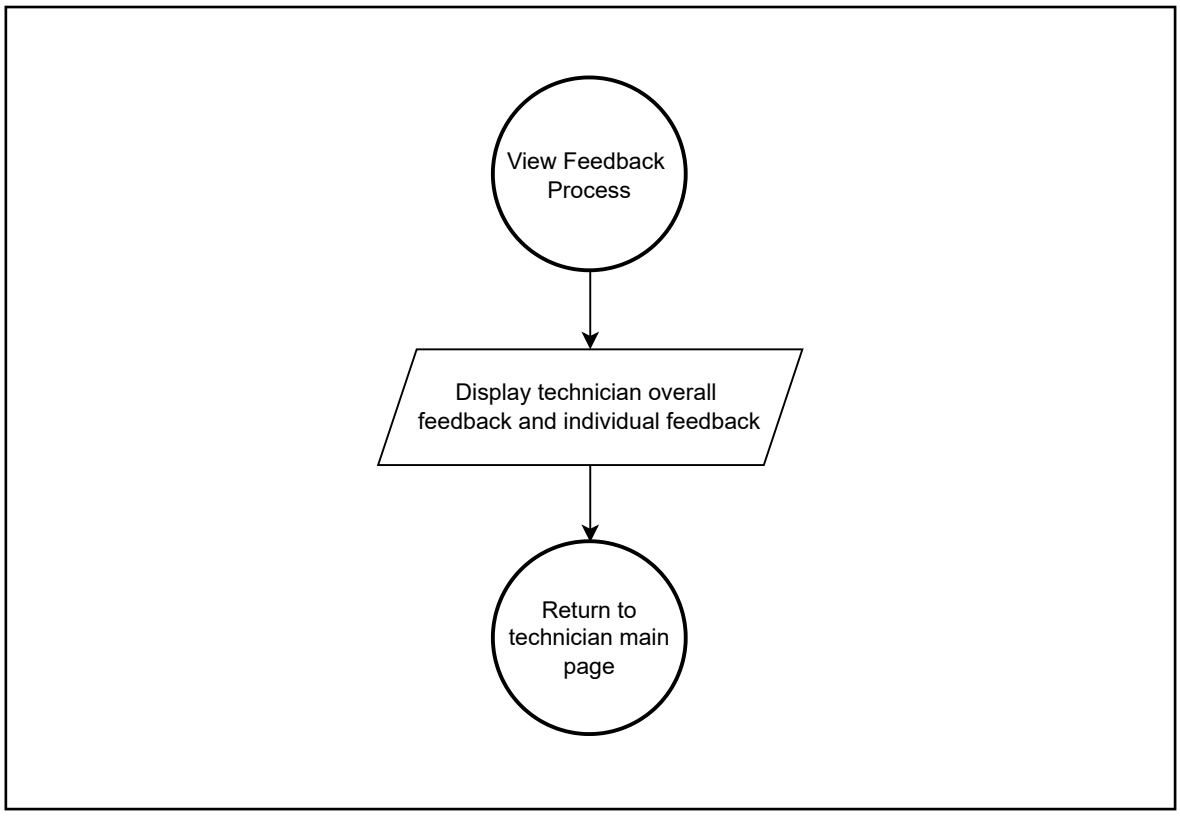

***Figure 11*: Flowchart for Technician View Feedback.**

Customer Main Page

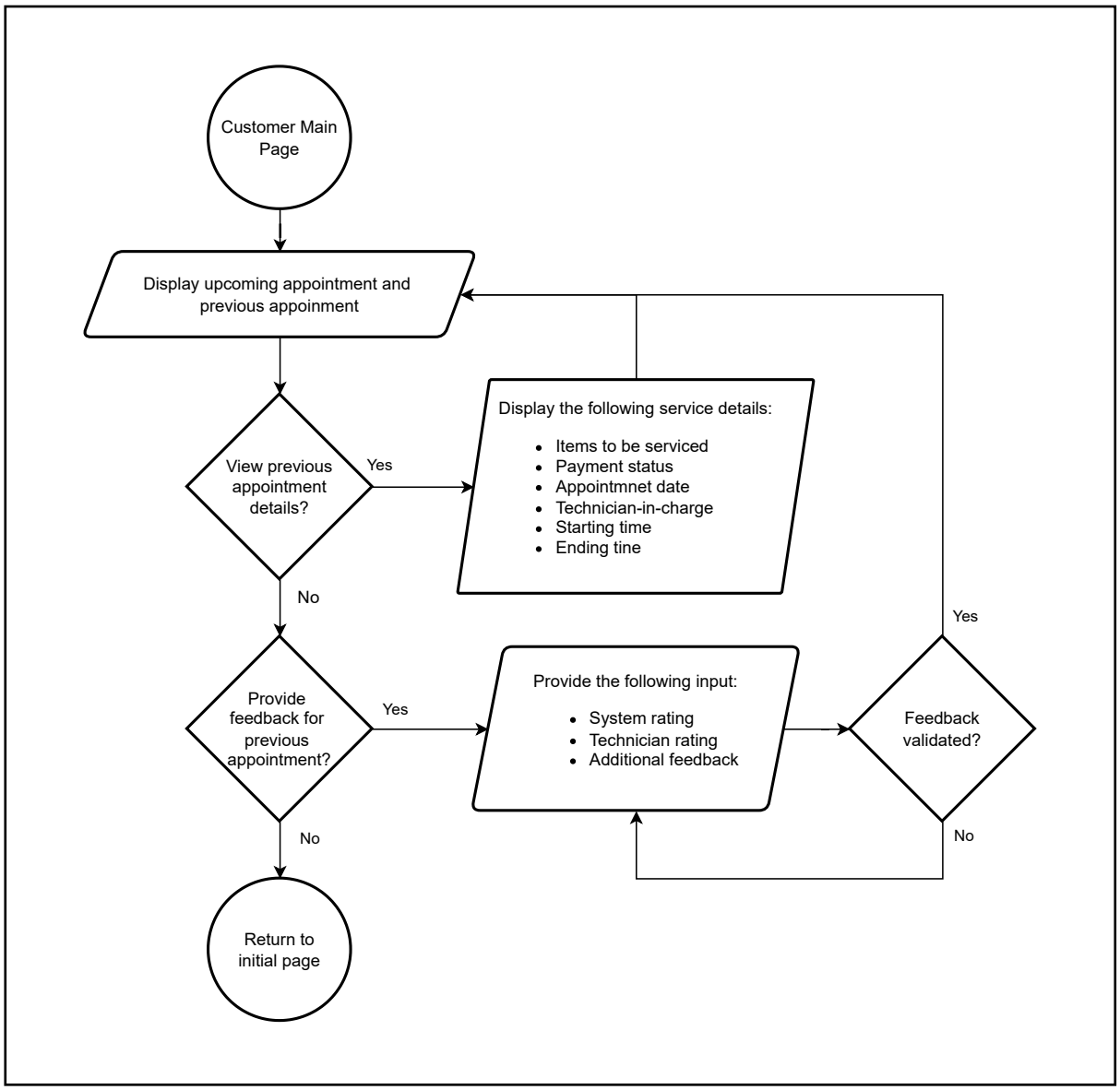

***Figure 12*: Flowchart for Customer Main Page.**

User Manual

Initial Main Page

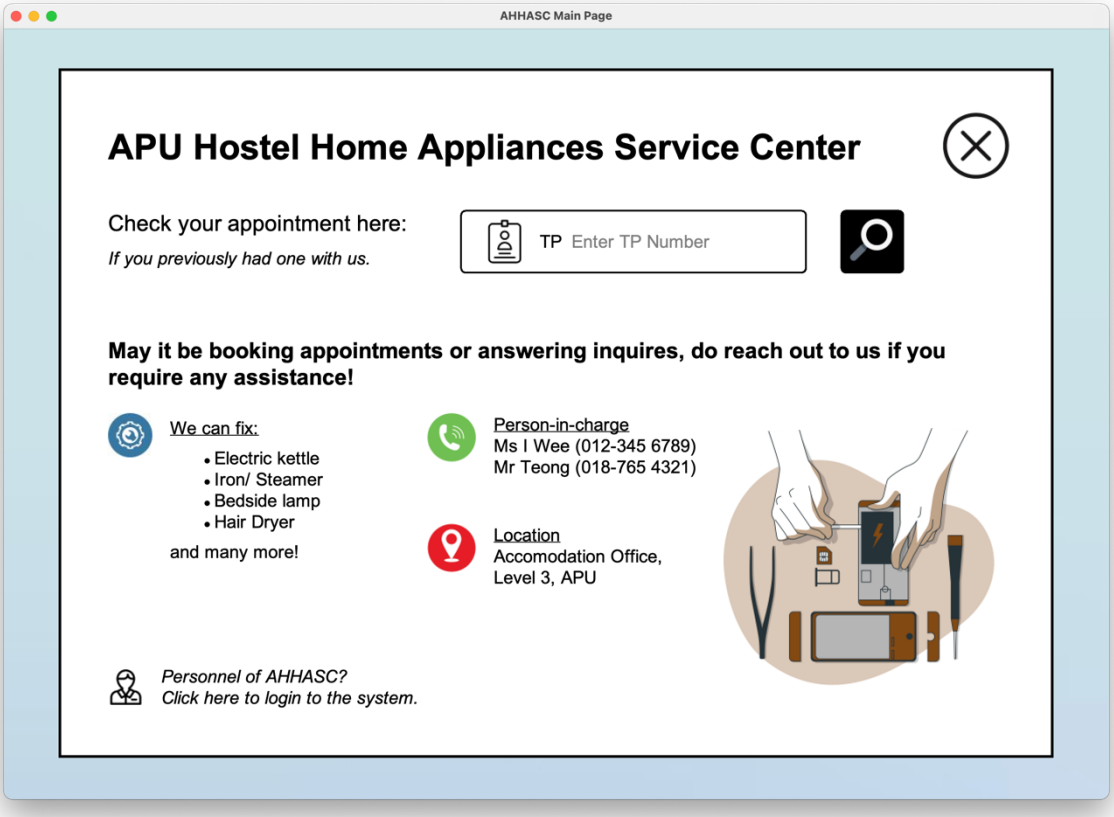

Figure 13: **Initial Main Page**

Upon running the system, users will be navigated to the initial main page, where the system will display all the details for the APU Hostel Home Appliances Service Centre (AHHASC), including the examples of items available for service, the contact details of the person-in-charge, and the location of the accommodation office. Accommodation tenants can utilize this information to contact the accommodation department personnel to place their bookings for any item services. However, the customers can view their previous and upcoming appointment details by entering their TP number inside the search bar and clicking on the search icon to proceed to the customer main page. On the other hand, managers and technicians of the AHHASC system will have to click on the text located at the bottom left of the system to be navigated to the login page.

Customer Main Page

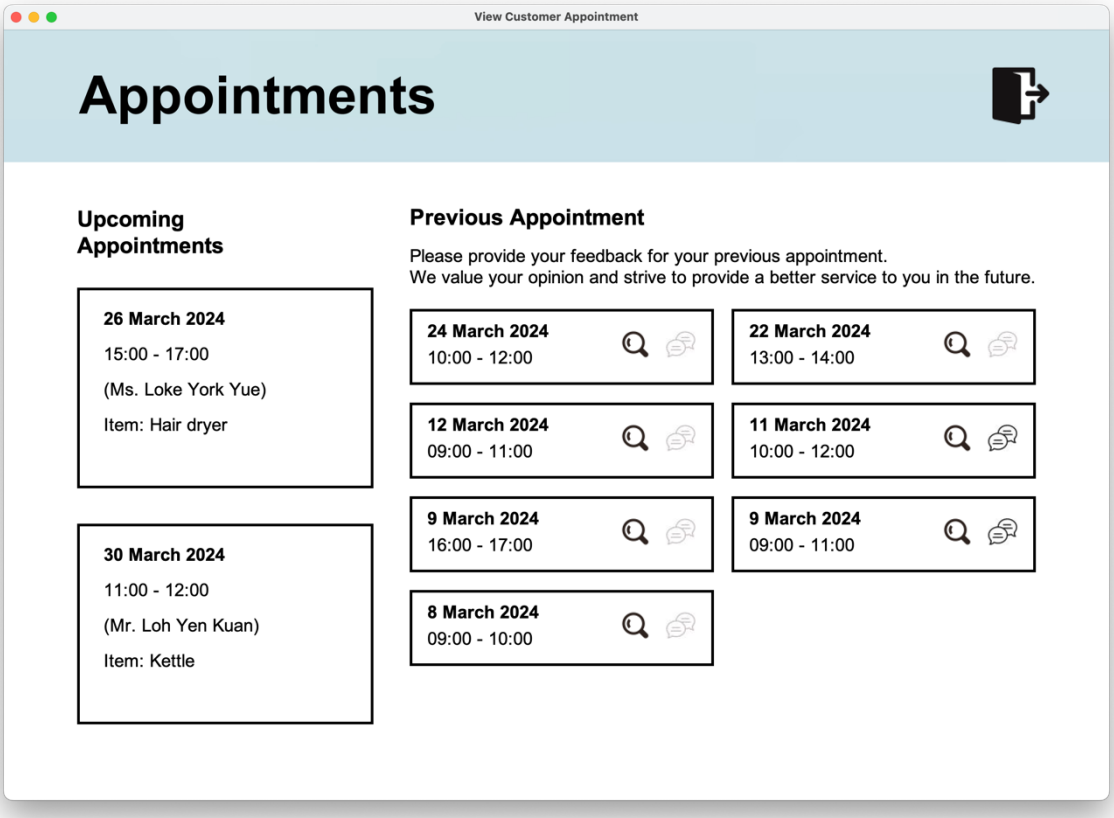

***Figure 14*: Customer Main Page**

After customers enter their TP number, if the account is recorded in the system, they will be redirected to the customer appointment page that displays the lists of their previous and upcoming appointments. On the left, customers can view their upcoming appointments with information, including the date, time, technician, and item that is involved in the item service. On the right, customers can view the list of their previous appointment history, arranged in reverse chronological order, with the option to obtain the details of the previous appointment by clicking on the view icon and providing feedback for all past appointments. However, once feedback is provided, the feedback icon will be greyed out, indicating that the customer cannot view the feedback details that have been inputted to the system.

Customer Feedback Input Page

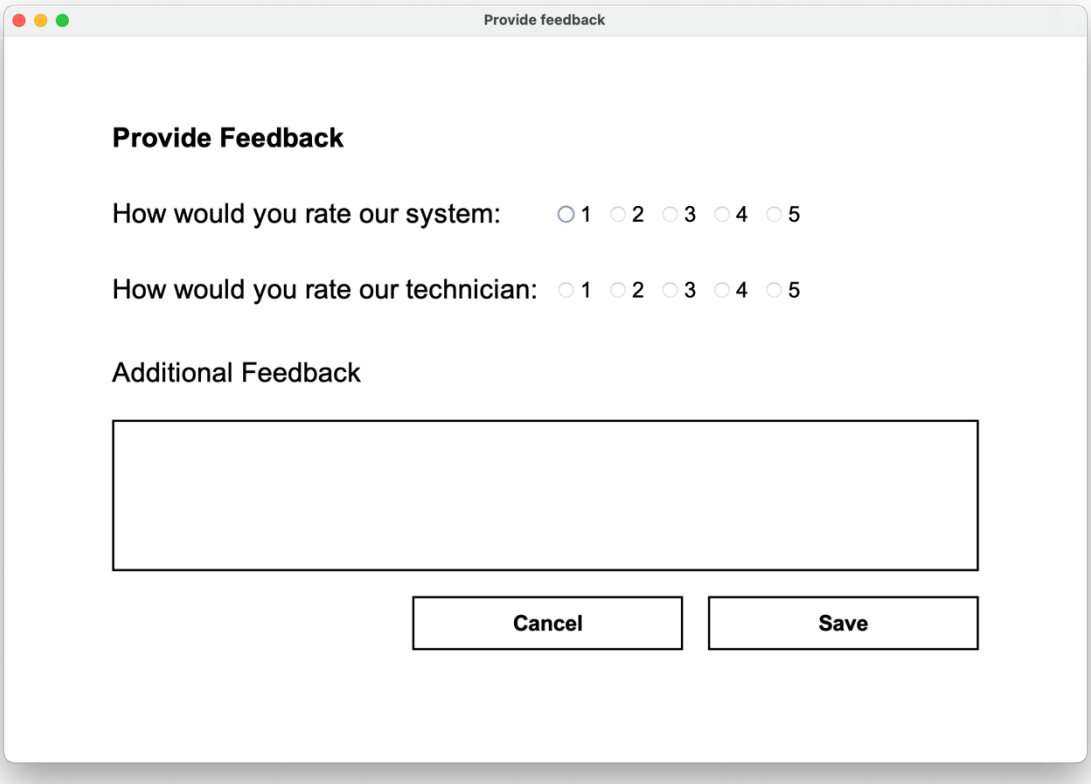

Figure 15: **Customer Feedback Input Page**

If customers wish to provide feedback for their previous appointments, they will be redirected to the Provide Feedback page after clicking the feedback icon. On this page, customers will provide ratings to the overall system and the technician in charge of their appointment. Customers can also provide additional feedback regarding their appointments by entering their opinions in the text box field. To ensure the feedback can be submitted successfully, customers must fill in all the ratings and information and click the save button. The input will then be stored in the system and displayed to the manager and technician so that the personnel of the accommodation center can identify the areas of improvement to provide a better service to their future customers.

Personnel Login Page

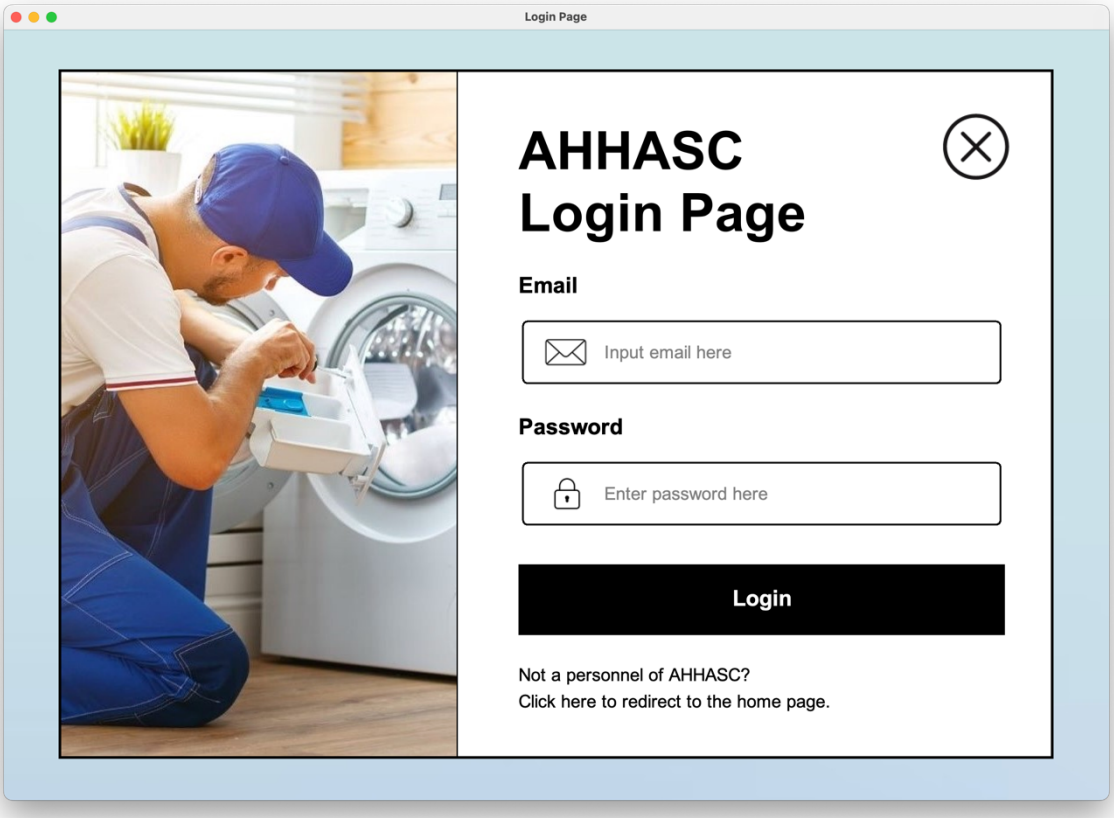

Figure 16: **Personnel Login Page**

For managers and technicians, after clicking on the login prompt text, they will be redirected to the personnel login page. Managers and technicians will have to input the correct credentials, including their email and password, to login to the system. The system will automatically identify the user type associated with the user and navigate them to the correct interfaces, where managers will be redirected to the manager main page, while technicians will be redirected to the technician main page, after clicking on the login button..

Manager Main Page

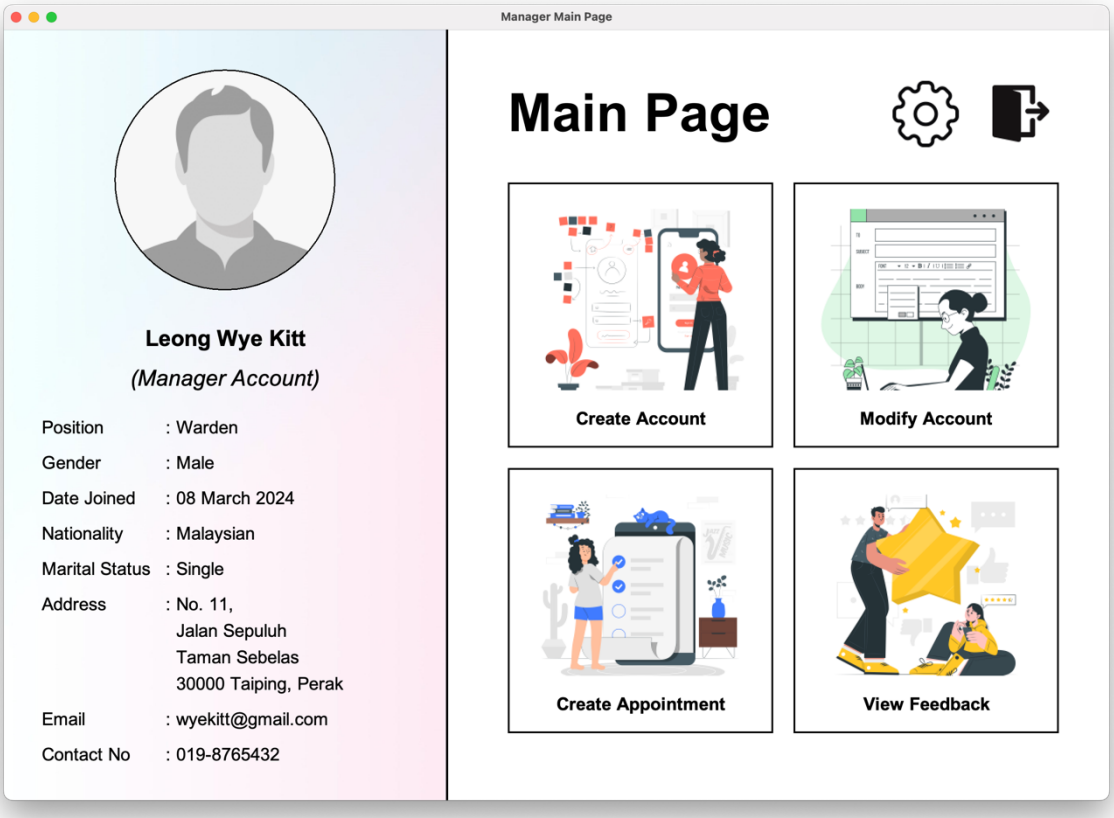

Figure 17: **Manager Main Page**

For managers, upon providing the correct credentials, the system will navigate them to the manager main page. On the left panel, the manager's personal details will be displayed, including their position, date joined, address, email, and contact number. If there is any inaccuracies in the information, they can edit them via the modify account feature of the system. The right panel displays all the available features available for the manager, such as creating and modifying accounts, book appointments and view overall feedback. There is also a gear icon located beside the logout button that allows managers to add the items available for service to the system. Based on their needs, managers will choose the desired functions and proceed to other pages.

Manager Create Account Page

If managers wish to create an account for other users, they will be redirected to the Create Account page after clicking the "Create Account" button. When the page loads, the system will prompt managers to select the types of accounts to create. Managers will then click the "Confirm" button and fill in all the details to create an account for the user type.

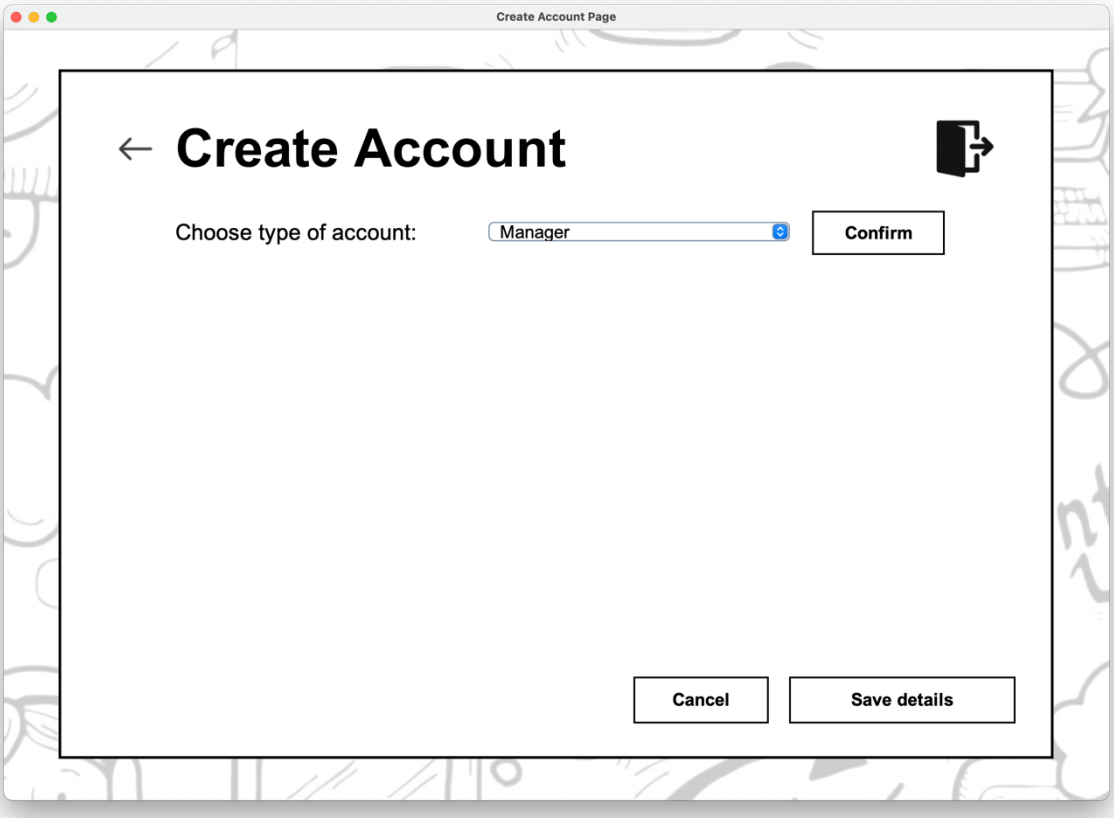

Figure 18: **Initial Manager Create Account Page**

For new manager and technician accounts, not only will managers be required to fill in the personal information, but they must also provide the login details of the new account, such as their email and password, by following the rules in the black text box. Managers must provide a new email that has yet to be used in the system to ensure successful account creation for managers and technicians. To save the details after filling in all the information, managers must click the save button, and the system will store all the information in text files. However, if managers wish to choose other types for the account, they will have to select the corresponding user type at the top of the page and click on the confirm button to fill in all the information again.

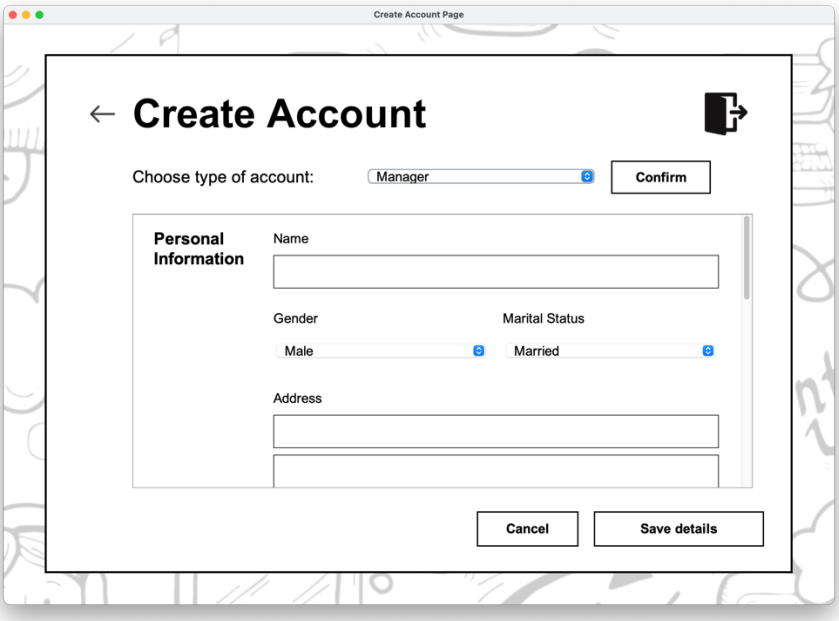

Figure 19: **Create Account Page to Create New Manager Account.**

On the other hand, to create a customer account, managers will not need to enter the login details for customers, as customers will be logging into the system by using their TP numbers. Hence, managers will only be required to fill in all the customer's personal information and click the save button to record all details. Similar to the account creation for managers and technicians, the TP number inputted for customers should not have been used by other customers on the system.

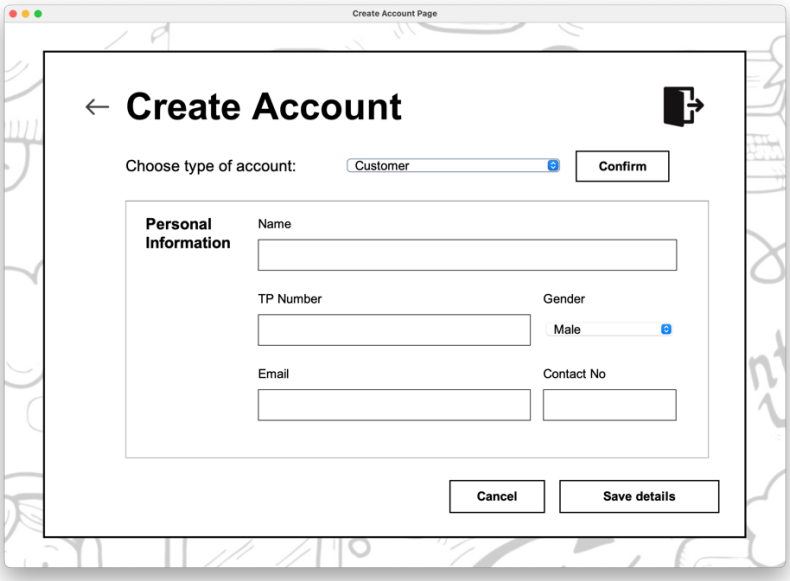

Figure 20: **Create Account Page to Create New Customer Account.**

Manager Modify Account Page

Managers can also modify the accounts created for the system by navigating to the modify account page. To make changes to a manager or technician account, managers must select the corresponding type of account and input the relevant email address. Suppose the account is available on the system. In that case, the system will automatically display the stored information so that managers will not need to manually type in all the data again, simplifying the entire process.

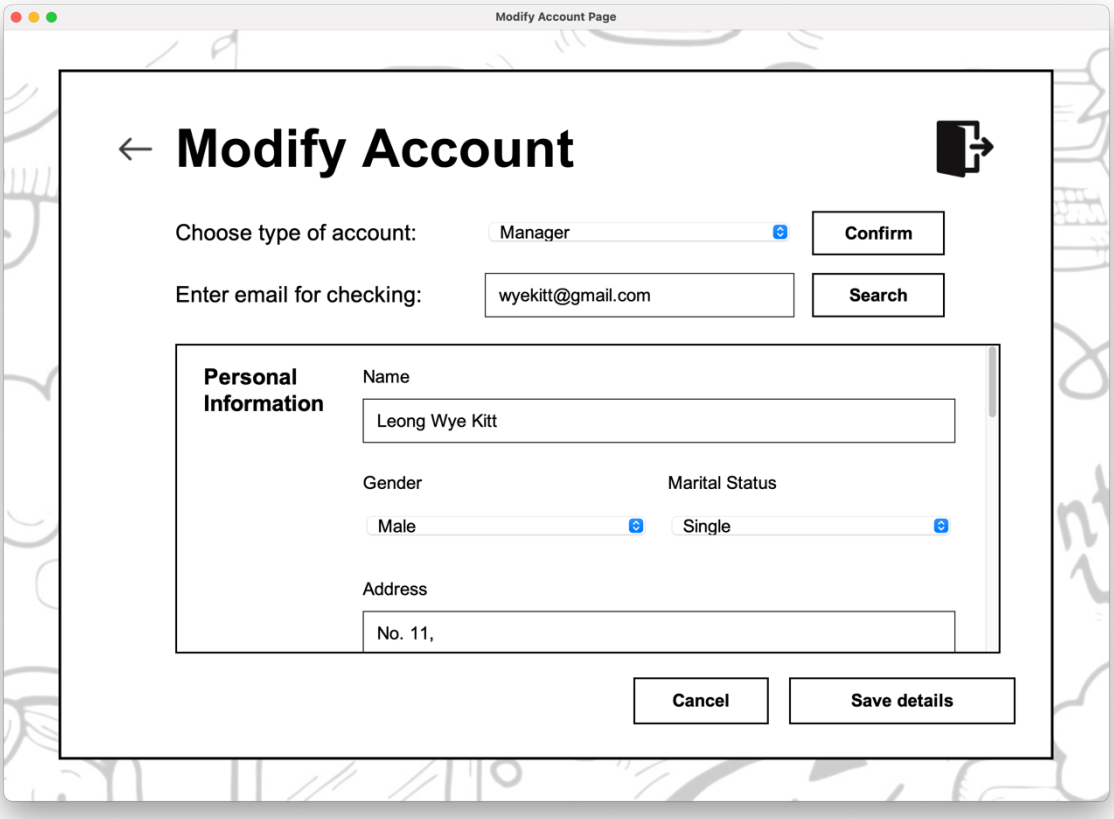

Figure 21: **Manger Modify Account Page**

Our system has also allowed managers to alter the account type of an existing manager or technician account. They can achieve this by selecting the corresponding account types via the drop-down panel and clicking on the confirm button to refresh the positions. However, if the technician account is previously associated with any appointments, changing the technician account to a manager account will delete all the related appointments from the system.

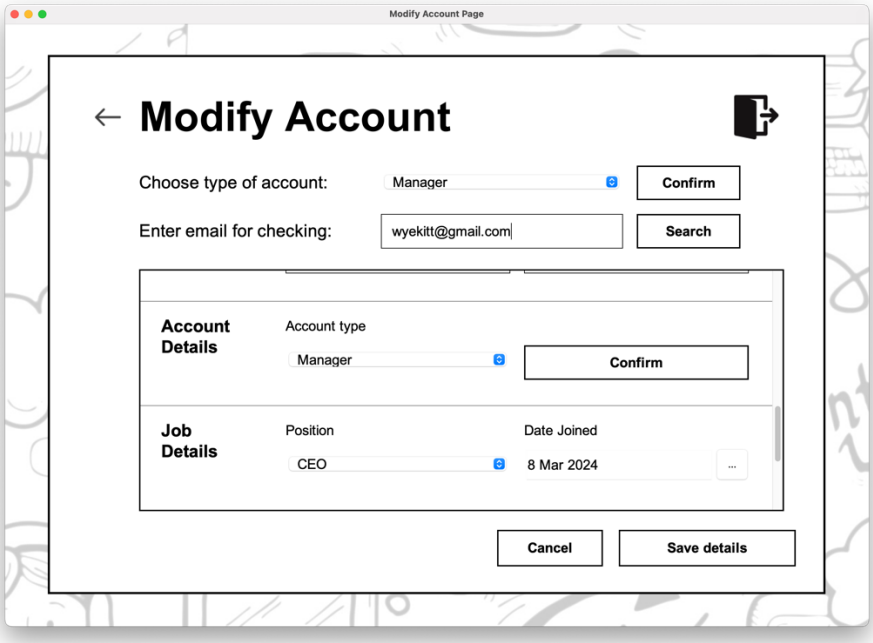

***Figure 22*: Account Details and Job Details Section for the Modify Account Page.**

Suppose managers wish to modify a customer account. In that case, they can only change customers' personal information but not the TP number, as the TP number is the credential that uniquely identifies each customer. This indicates that once a customer account is created, its TP number cannot be modified anymore, so managers should always double-check the credentials before creating a customer account.

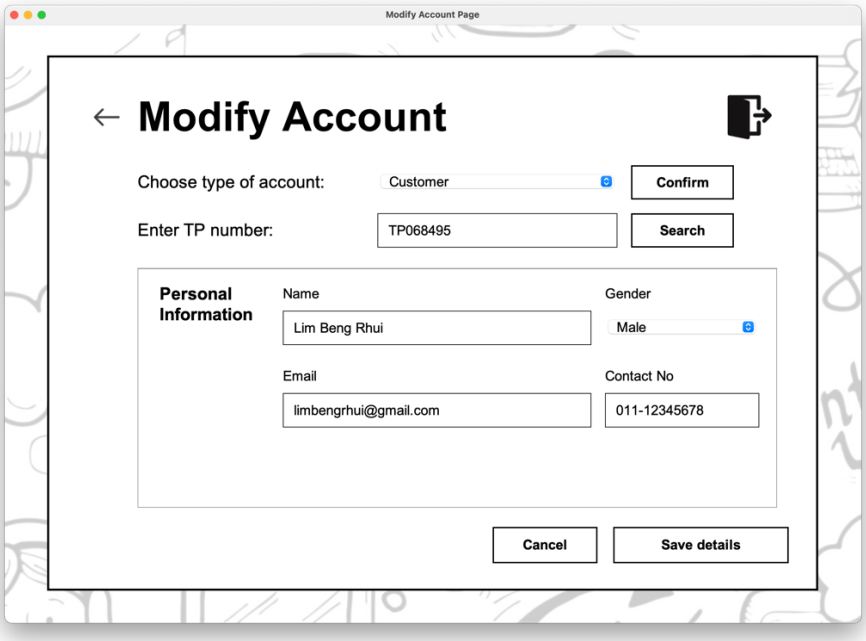

***Figure 23*: Manager Modify Account Page to Alter a Customer Account.**

Manager Appointment Booking Page

If managers would like to book an appointment for a customer, they shall redirect the system to this page. After entering the customer TP number and identifying the customer, managers can select the technician-in-charge before choosing the items to be serviced via the drop-down menu. Our system has also integrated a feature that allows managers to view the technician's schedule, which can be assessed after selecting the correct technician and appointment date. Upon filling in all the information and pressing the place appointment button, customers and technicians will create and view the appointment on their respective pages.

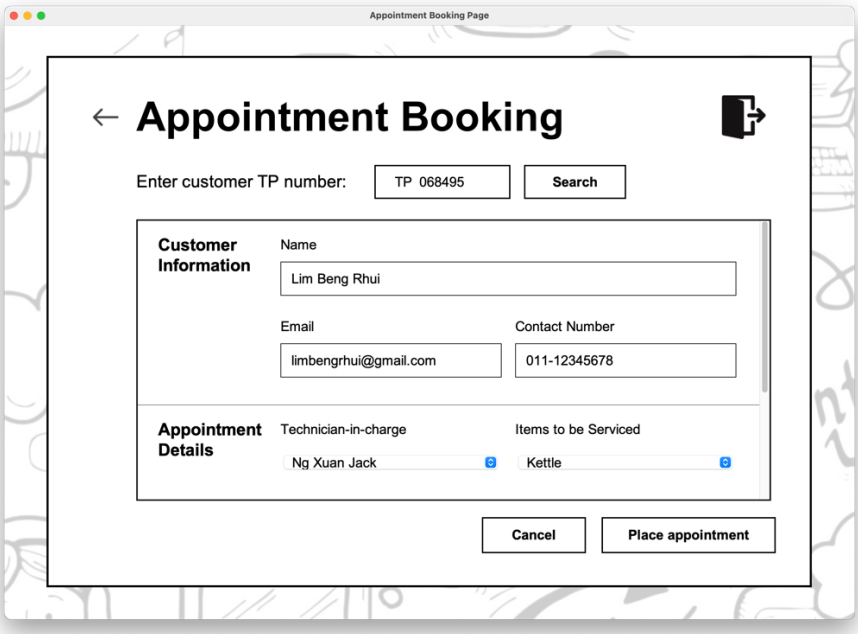

Figure 24: **Manager Appointment Booking Page.**

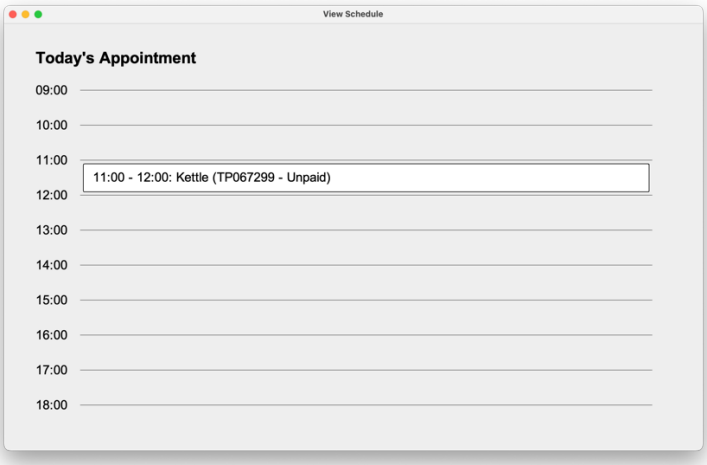

Figure 25: **Schedule Displayed from the Manager Appointment Booking Page.**

Manager View Feedback Page

Suppose managers wish to view the overall feedback provided by the customers. In that case, they can navigate to the view feedback page, where the system will calculate the average ratings from the users and outline all individual feedback provided by each customer. Managers can then scroll through the individual feedback to view the comments provided by customers regarding the services offered by AHHASC.

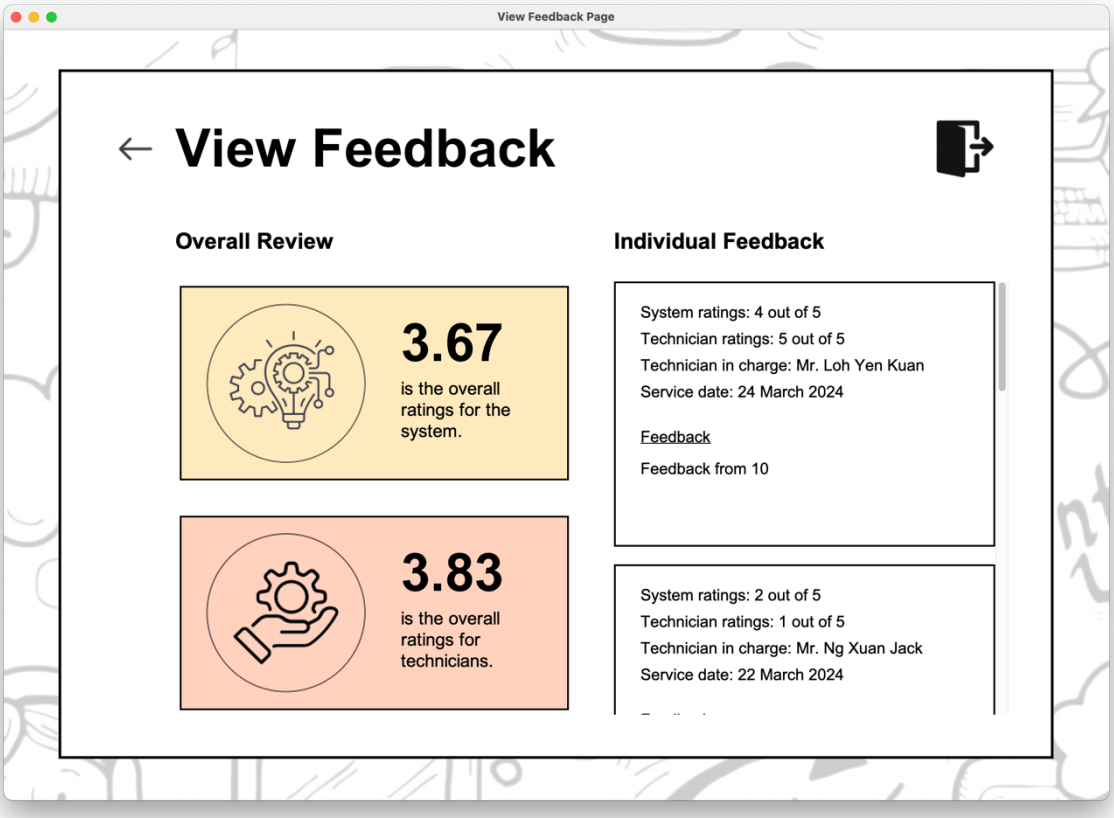

***Figure 26*: Manager View Feedback Page**

Manager Add Item Page

To add items available for service to the system, managers can click on the gear icon on the manager main page, and the manager add item page window will be displayed. Managers will need to input the item name and price, and the item will automatically be added to the system if the item name does not clash with the names of other items in the system.

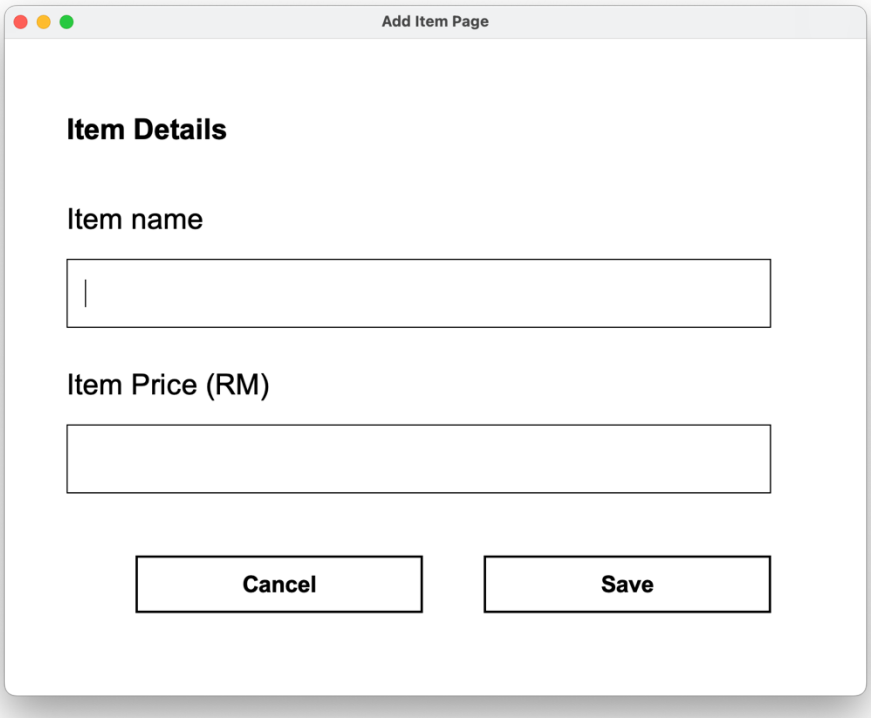

Figure 27: **Manager Add Item Page.**

Technician Main Page

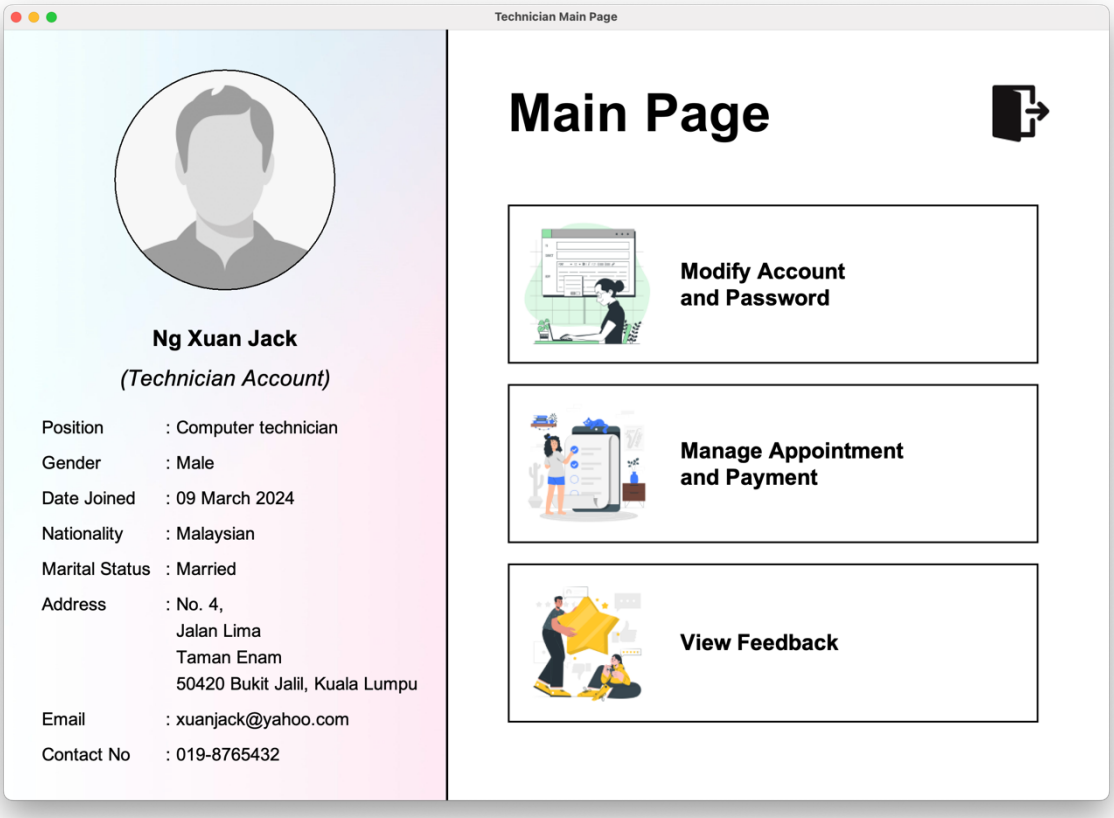

Figure 28: **Technician Main Page**

After entering the correct credentials, technicians can navigate to the technician's main page. Like the manager's main page, the left panel displays the technician's details. In contrast, the right panel shows all available features for the technician, including modifying the technician’s account and password, managing all appointments and payments, and viewing customer feedback. If they wish to exit this page, they can click the logout button at the top right corner.

Technician Modify Account Page

Technicians can modify their personal information and password by navigating to the modify account page. After editing all the details that are displayed automatically, technicians can click on the save button to update the system. One point to note is that if the technicians do not wish to modify their password, they will not need to enter anything at the password field and leave it blank. If technicians decided to abandon the editing process, they can choose to return to the technical main page by clicking on the back button located at the top left corner of the page.

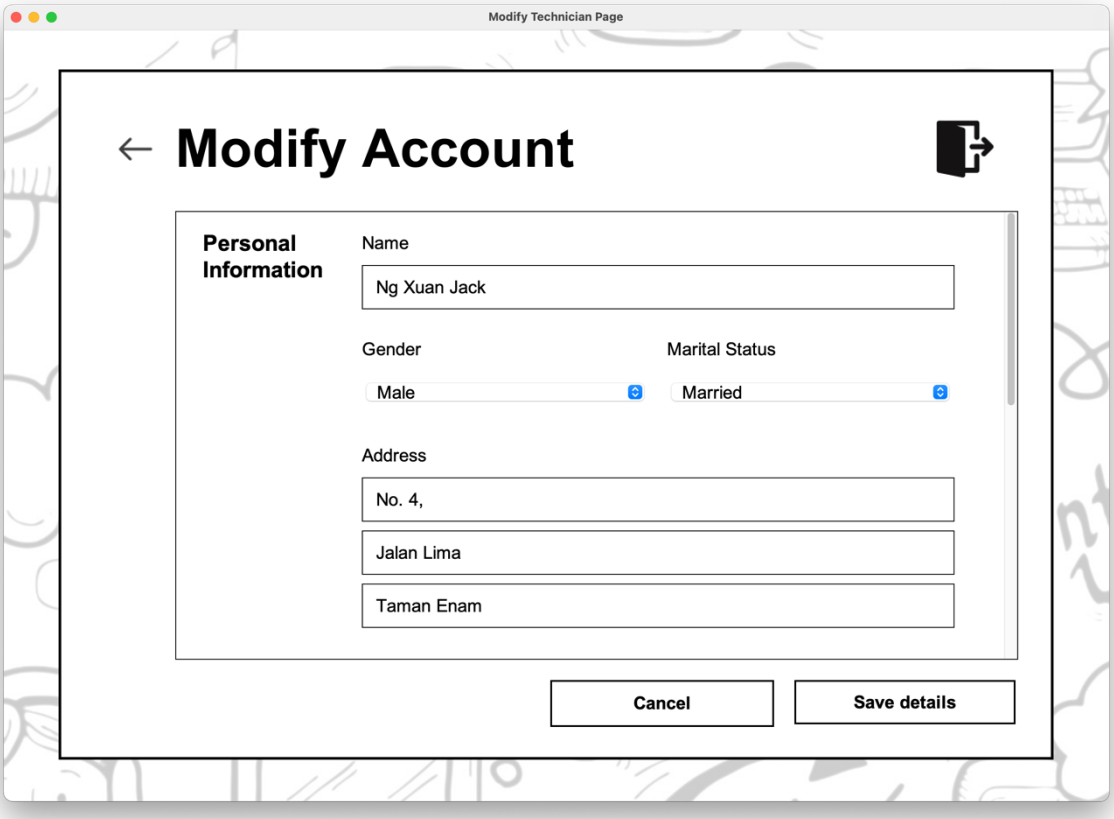

Figure 29: Technician Modify Account Page

Technician Manage Appointment Page

To view information related to appointments, technicians shall click on the manage appointment and payment button from their main page and be redirected to the manage appointment page. They will be displayed with a schedule that depicts all appointments for the current day. By clicking on the switch view button, technicians can view a table that outlines all appointments with details, including the option to change the payment status to paid or unpaid by checking the check box at the end of each record. Technicians should remember to click the save button at the bottom right to ensure the information is updated.

Figure 30: **Manage Appointment Page Displaying Appointment for Current Day.**

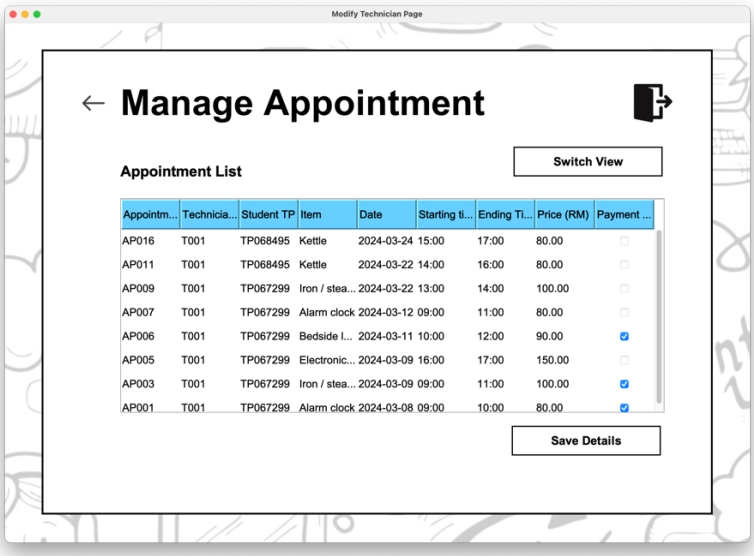

Figure 31: **Manage Appointment Page Displaying Lists of Appointments.**

Technician View Feedback Page

The technician will be navigated to the feedback page to view the overall technician review and individual feedback by clicking the view feedback button. The overall review is about the overall score for the service provided by the technician, while the individual feedback depicts each comment provided by the customer to the specific technician. However, each technician can only view their feedback and can scroll down to view many more individual feedback.

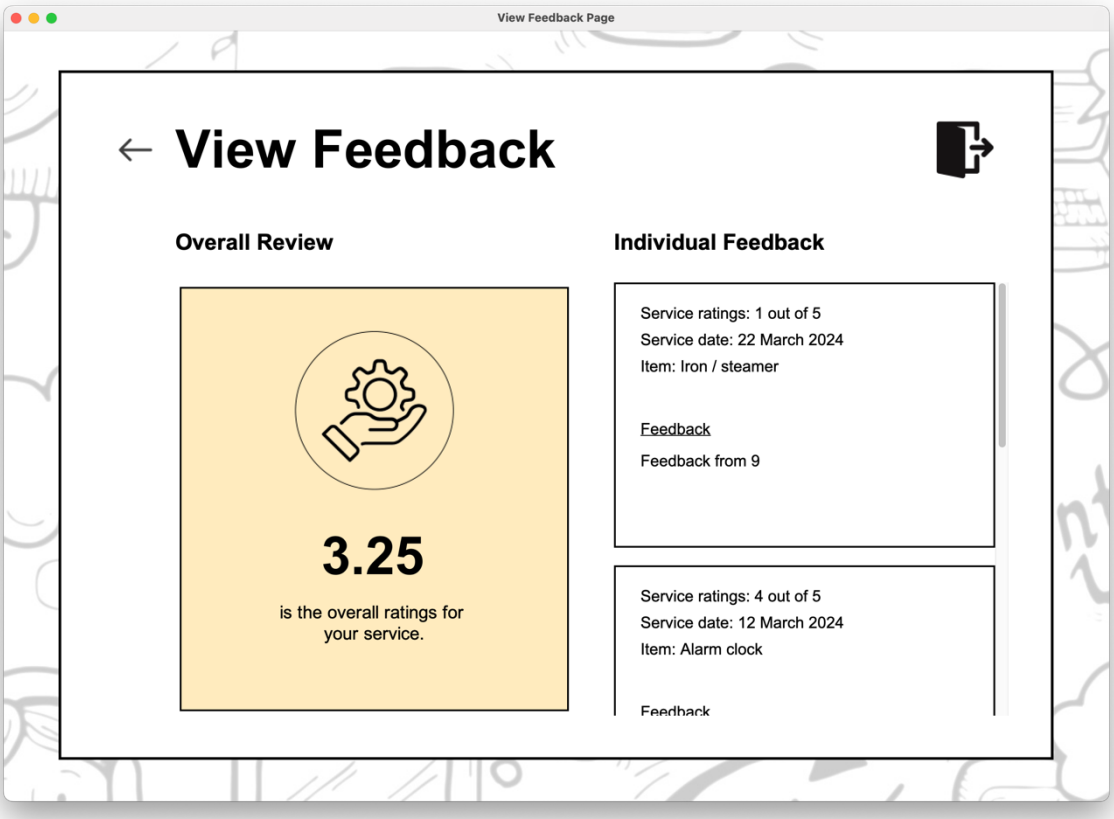

Figure 32: View Feedback Page

Object Oriented Concepts with Sample Code

Encapsulation

Often implemented in object-oriented programming, encapsulation refers to the concept of limiting programmers' access to directly referencing the variable declared in a class (Sumo Logic, 2024). When encapsulation is implemented, the internal representation of the data is hidden, but it may be altered using methods defined in the class, such as getters and setters. Such a mechanism is implemented as having the data hidden away from programmers so that they will not introduce any program errors invoked due to the illegal modification of data from the class, hence promoting data integrity.

```
public class Appointment {

    private final static String[] appointmentTime = {"09:00", "10:00", "11:00", "12:00",
"13:00", "14:00", "15:00", "16:00", "17:00", "18:00"};

    private static ArrayList<Appointment> overallAppointmentList = new ArrayList<>();

    public static String[] getAppointmentTime() {
        return appointmentTime;
    }

    public static void setOverallAppointmentList(ArrayList<Appointment> list) {
        overallAppointmentList = list;
    }
}
```

***Figure 33*: Sample Code to Demonstrate the Concept of Encapsulation.**

Obtained from the Appointment class, the code demonstrates the concept of encapsulation by implementing modifiers, which are the *public* and *private* keywords, together with the concepts of *getters* and *setters*. Modifiers refer to the keywords that control the accessibility of the variables and methods within the project (Great Learning Team, 2023). For instance, the *public* keyword allows the instance to be accessed by any other codes in the package, while the *private* keyword limits the instance where it can only be accessed by the codes within the same class. In our case, our team has declared two variables to be stored in the Appointment class, which is a string array called "appointmentTime" that stores the available time slots to book an appointment and an array from the ArrayList class called as "overallAppointmentList" that stores the entire list of appointment details from the text file.

Setting these two variables as *private* limits programmers from directly referencing them when they need to use the information from the variables, which helps prevent any unexpected modification towards the data stored in the variables. As an example, since we have outlined the assumption that appointments are only available from 9:00 a.m. till 6:00 p.m., by using the private keyword for the "appointmentTime" variable, programmers cannot directly modify the string array as they wish, which ensures that the appointment time slots are consistent throughout the system.

Getters and *setters* are then implemented to obtain the value from the "appointmentTime" array and set the values to the "overallAppointmentList" array. As an example, the getter for the "appointmentTime" array, called the "getAppointmentTime()" method, is implemented in the InformationPaneComponent class to create a combo box for managers to select a time when placing appointments. Our team has used the *getter* to obtain the lists of the time slots and saved it in another string array known as the "appointmentTimeSlot" so that we can pass the string array into the constructor of the JComboBox to build the combo box with the elements in the string array as user choices. By using *getters*, if the programmer implements any commands that remove a particular time slot from the "appointmentTimeSlot" array, the initial string array from the Appointment class, "appointmentTime," will not be affected, hence ensuring that the other parts of the code that relies on the "appointmentTime" array will not get affected by the change invoked by programmers. Such a scenario promotes code maintainability and flexibility, as changes to the internal workings of the classes can be made without affecting the code that uses them.

```
public class InformationPaneComponent extends JPanel {

    JComboBox<String> appointmentStartAD, appointmentEndAD;

    public JPanel appointmentDetails() {

        String[] appointmentTimeSlot = Appointment.getAppointmentTime();

        appointmentStartAD = new JComboBox<>(appointmentTimeSlot);
        appointmentStartAD.setFont(Asset.getBodyFont("Plain"));
        appointmentStartAD.setBackground(Color.WHITE);

        appointmentEndAD = new JComboBox<>(appointmentTimeSlot);
        appointmentEndAD.setBackground(Color.WHITE);
        appointmentEndAD.setFont(Asset.getBodyFont("Plain"));
    }
}
```

***Figure 34*. Implementation of Getters for the "appointmentTime" Array.**

Abstraction

Abstraction refers to hiding the implementation details of a code so that programmers can focus on retrieving the critical information without needing to understand the underlying concepts and ideas involved in the components (Bloch, 2018). Having abstraction allows programmers to simplify the implementation of components by disregarding the unimportant features in the methods, resulting in lower complexity during coding development (Sangai, 2024 (Schildt, 2014)). Abstraction can be implemented in several ways, such as constructing a superclass that defines a generalization form so that the subclasses can be further modified to provide each with unique attributes and methods (GFG, 2017). Another method would be coming up with methods that take in several arguments and process them to return a specific value used in the system.

```
public class Appointment {

    public static boolean checkAppointmentAvailability(String technicianName, LocalDate
    date, LocalTime startingTime, LocalTime endingTime) {

        TextFileOperationsComponent.readAppointment();
        boolean available = true;

        String technicianID = Technician.getIdFromName(technicianName);

        for (Appointment appointment: Appointment.getOverallAppointmentList()) {

            if (appointment.technicianID.equals(technicianID) &&
            appointment.date.isEqual(date)) {

                if (startingTime.isBefore(appointment.startingTime)) {
                    if (endingTime.isAfter(appointment.startingTime)) {
                        available = false;
                        break;
                    }

                } else {
                    if (startingTime.isBefore(appointment.endingTime)) {
                        available = false;
                        break;
                    }
                }
            }
        }

        return available;

    }
```

***Figure 35*. Sample Code to Demonstration the Concept of Abstraction.**

An example of implementing abstraction in our code would be the "checkAppointmentAvailability" method defined in the Appointment class. This method aims to check the availability status of a specific technician on a particular date within a time slot. In the code, after reading the overall list of appointments from the text file, we have defined a boolean variable called "available" to record the availability status. After retrieving the technician ID based on the technician's name, we create a for loop to evaluate different types of conditions based on the current appointment details of the technician. Under the conditional statement that filters the list of appointments so that only appointments related to the technician will be evaluated, we have then declared two conditions: the starting time provided as an input is before the starting time recorded in the appointment details, but the input ending time is after the starting time of the recorded appointment (inputted ending time falls between the starting time and ending time of an appointment), and the inputted starting time is after the starting time recorded in the appointment but the inputted starting time is before the end of the recorded ending time in the appointment (starting time falls between the starting and ending time of the appointment). If one of the two conditions is satisfied, the available variable is marked as false, indicating that the technician is unavailable at the provided time slot.

Suppose the programmer were to write all the codes whenever they wish to evaluate the availability of a technician. In that case, the code for the system will inevitably be very lengthy and repetitive. Such a situation might also introduce issues if the variables are not passed in correctly. If there is a bug within the lines of codes, it would be very tedious for the programmers to amend the code one by one, leading to inefficiency during the coding process. Therefore, a better approach would be developing a method to check the availability of technicians, where the method would return a boolean value of true and false that indicates the technician's availability upon passing in the correct arguments. Such a method simplifies the coding process as programmers only have to type one statement to evaluate the technician's availability. Moreover, the method rigorously helps to improve the code's readability when the appropriate name is used to name the methods (Martin, 2008). Such is the power of the abstraction concept, where programmers who will be utilizing the checkAppointmentAvailability() method in their programs can quickly implement the method to check for technician's availability without needing to spend time in developing the algorithm to evaluate the availability of technician, yielding a much convenient and efficient process during the development of the system.

Inheritance

Inheritance refers to the process by which one item acquires attributes from another, which plays a significant role in object-oriented programming as it validates the notion of hierarchical categorization. Without hierarchies, programmers would have to explicitly define the qualities of each object, which yields inefficiency when multiple objects share similar attributes. Therefore, to reduce repetition and increase the maintainability of codes, the concept of inheritance is introduced in object-oriented programming, where classes, which are known as the subclass or the child class, can inherit attributes and methods of another class, which is known as the superclass or the parent class (Sierra & Bates, 2005). The relationship between superclass and subclass is a one-to-many relationship, where a superclass can have multiple subclasses, but each subclass can only have a superclass. With the implementation of inheritance, not only will programmers not have to declare the attributes and methods for the subclasses, but programmers could further append additional attributes and methods for these classes to make them specialized for a particular usage.

```
public class User {

    String name, gender, email, contactNumber;

    public User(String name, String gender, String email, String contactNumber) {
        this.name = name;
        this.gender = gender;
        this.email = email;
        this.contactNumber = contactNumber;
    }
}
```

***Figure 36*: Code to Demonstrate the Parent Class in the Inheritance Concept.**

```
public class Student extends User {

    String tpNumber;

    public Student(String tpNumber, String name, String gender, String email, String
contactNumber) {
        super(name, gender, email, contactNumber);
        this.tpNumber = tpNumber;
    }
}
```

***Figure 37*: Code to Demonstrate the Child Class in the Inheritance Concept**

In Java, the concept of inheritance is applied using the "extends" keyword (Schildt, 2016). An example would be the class inheritance relationship between the User and Student class. Since students, or customers, are considered a type of user, our team has included the "extends" keyword when defining the Student class. This makes the Student class the subclass that inherits all the attributes and methods from the User class, which is the superclass. Therefore, we would not have to declare all the user's attributes, including name, gender, email, and contact number, in the Student class. We were only required to declare the attributes specially catered for students, which is the TP number that uniquely identifies each student, which is not available on other account types like managers and technicians. However, when building the constructor, we must include the "super" keyword at the first line so that the parent class, which is the User class, is properly initialized before we initialize our Student subclass. If the superclass is not initialized, the subclass will not be able to utilize the attributes and methods in the superclass, which might yield potential errors if we try to employ the attributes from the subclass during the coding process. The inheritance relationship between the User superclass and Student subclass enabled us to model real-life relationships, as well as allow our team to develop unique subclasses without needing to redeclare similar attributes across different classes.

Modularity

Modularity refers to partitioning the software environment into different parts or modules to make the coding process more organized and optimized. As we have divided each part of our program into distinct classes or methods, implementing modularity during the software development process will aid in reducing the coupling effect, which refers to the scenario where a minor bug or error in the code might lead to massive complications when we attempt to fix the mistakes. Not only does modularity improve the readability of codes due to the organization of codes, but it also assists developers in performing functionality testing on the go while the software development process is ongoing simultaneously due to the advantages brought by loose coupling, where components do not heavily depend on the internal structure of another element if the modularity concept is applied (Vats, 2022).

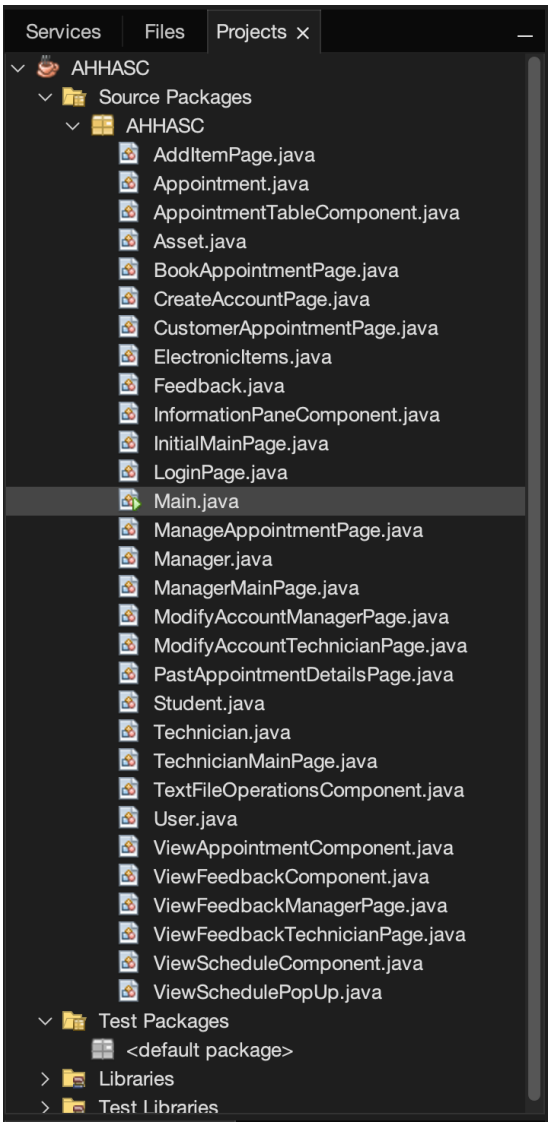

***Figure 38*: Modularity of Classes in the AHHASC System.**

Foreseeing the benefits brought by modularity to our project, our team has tried to break down each functionality and object in our system into different classes, as shown in Figure x. Such a scenario not only allows all members of our team to develop the GUI of each page individually without needing to worry about the problem of integrating all components after everything has been built but also enables us to easily debug for errors by narrowing the search scope down when identifying the source code that triggered the error. For instance, when developing the AHHASC application, our team separated the codes into classes when we created the GUI for managers, technicians, and customers for simultaneous development to speed up the system development process.

```
public class Asset implements MouseListener {

    public static Font getBodyFont(String type) {...}

    public static Font getNameFont(String type) {...}

    public static Font getTitleFont() {...}

    public JLabel generateImage(String pictureName) {...}

    public JPanel generateRoundedRectangle(int width, int height, int roundedCorner, int
    borderThickness) {...}

    public JPanel generateFillRoundedRectangle(int width, int height, int roundedCorner,
    int borderThickness, Color color) {...}

    public JPanel generateRoundProfile(String filePath, int radius) {...}

    public JPanel drawLine(int startX, int startY, int endX, int endY, int
    strokeThickness) {...}
```

***Figure 39*: Source Code to Demonstrate the Concept of Modularity.**

Aside from having classes, our team has also broken down the components in each class into methods, demonstrating the concept of modularity as each method deals with specific tasks. An appropriate example would be the Asset class, where we have separated the process of building each GUI component into different methods. We have catered to three different methods for generating various types of fonts for consistency and developed multiple methods that assist us in drawing 2D shapes like rounded rectangles, lines, and profile pictures in the shape of a circle. Via the breaking down of classes into methods catered explicitly for a particular purpose, not only can our team reuse these components when we attempt to develop the GUI for our system, but it also allows our team to easily modify the appearance of the GUI assets without needing to manually edit the codes one-by-one within the classes for developing GUIs.

Additional Features

Our team has also introduced a few additional features, not only in terms of the functions of our system but also extra GUI concepts that are not discussed during class.

Interface for Customers

Our team has introduced an extra interface for customers to view their upcoming and previous appointments, along with details like the technician, items, date, and time of the appointments. Furthermore, customers could provide feedback for their earlier appointments via the system without requiring them to inform the manager regarding their opinions towards the AHHASC system and the service provided.

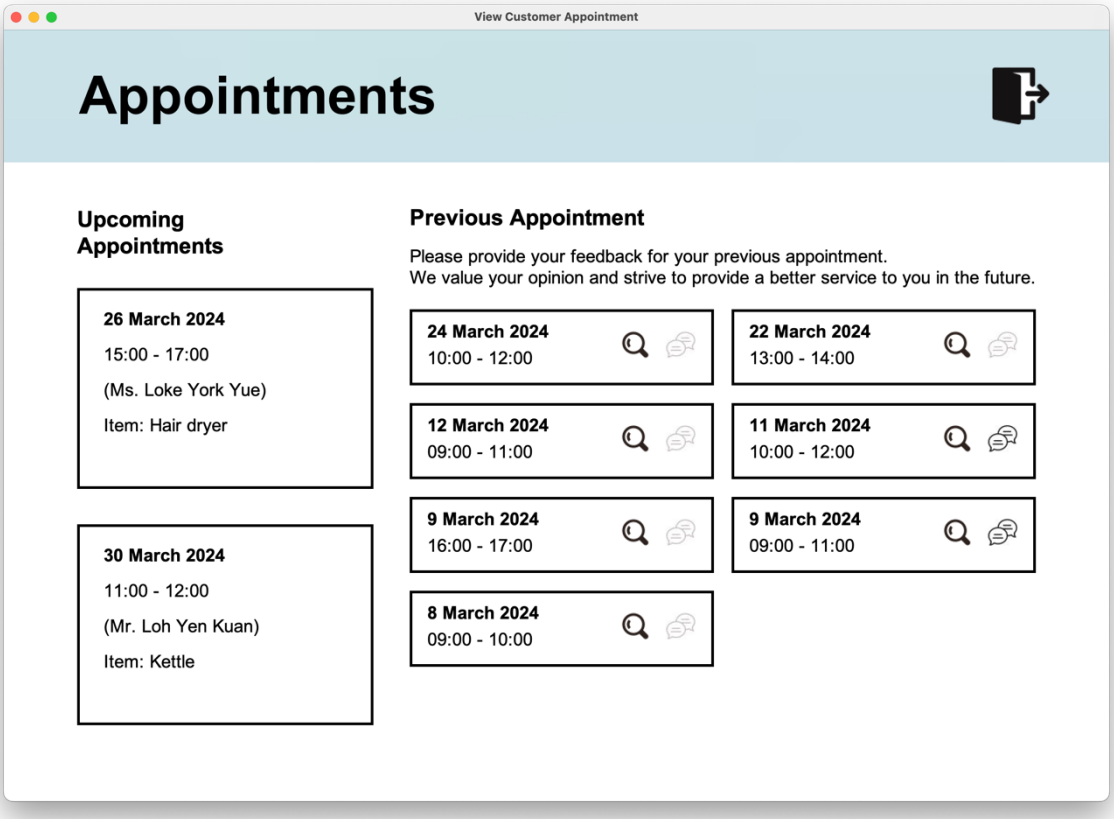

***Figure 40*: Interface for Customers.**

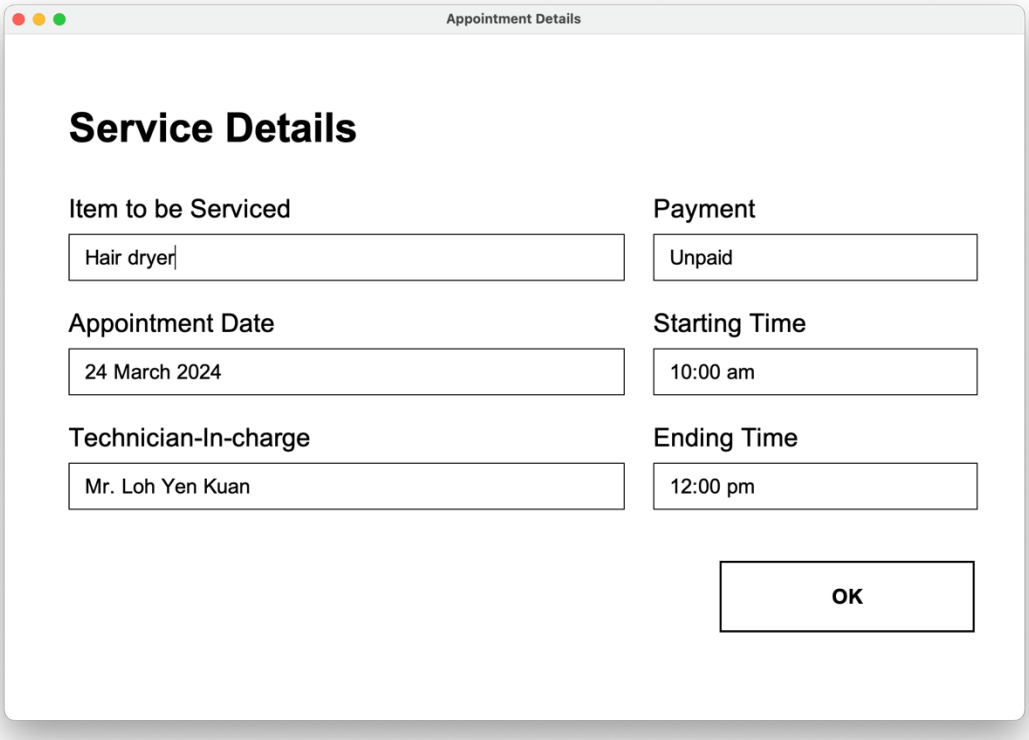

Figure 41: Details of Customer's Previous Appointment.

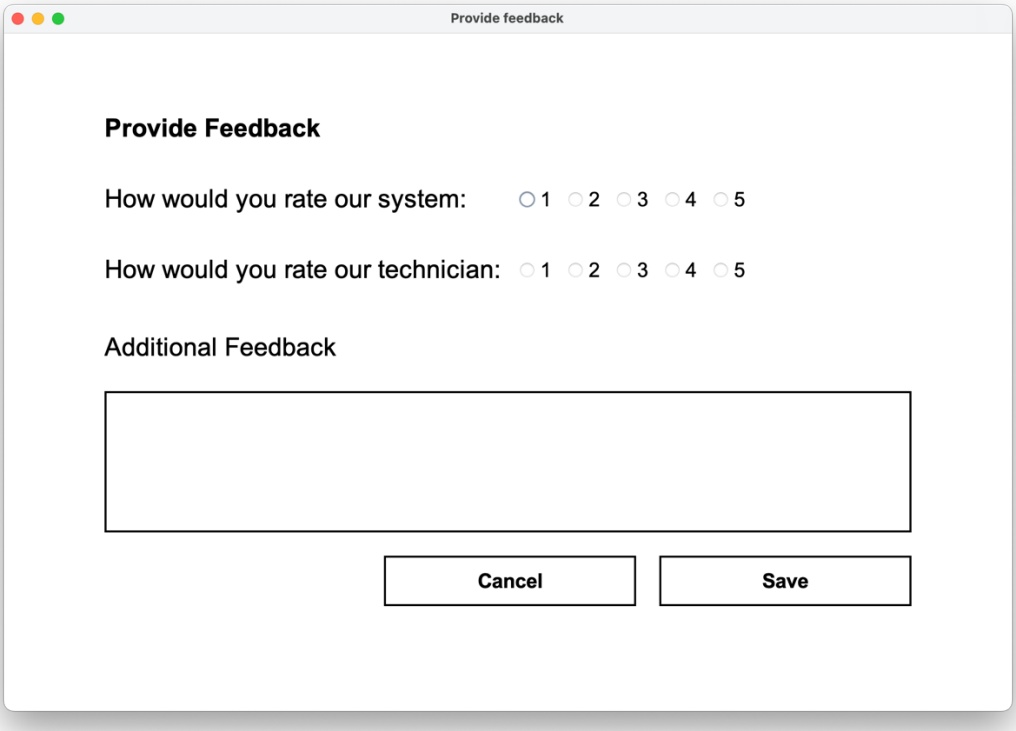

Figure 42: Interface to Provide Feedbacks.

Additional GUI components

JScrollPane

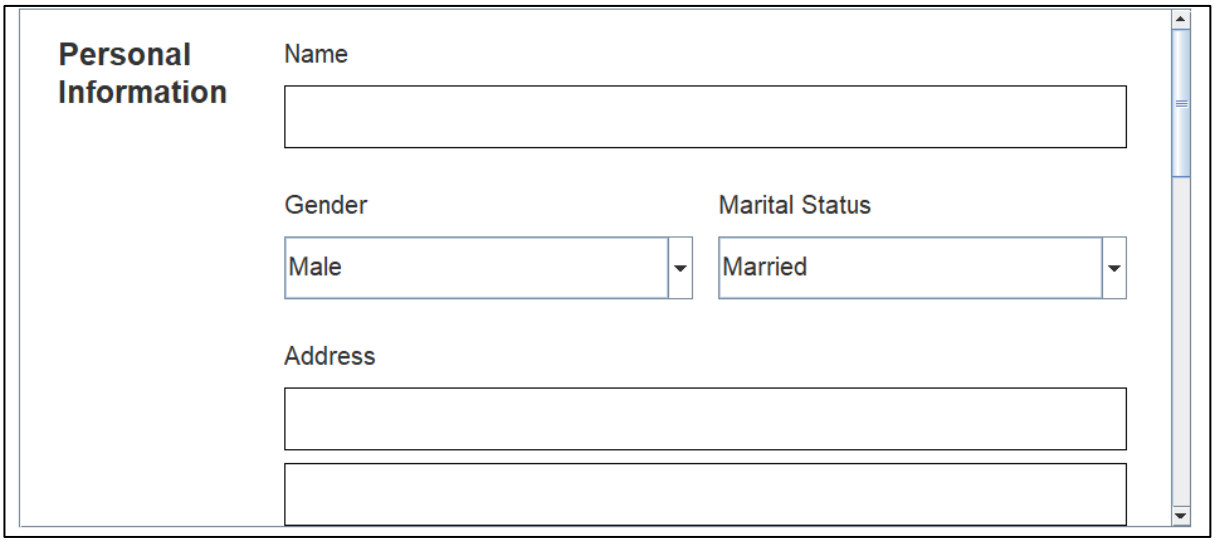

***Figure 43*: Implementation of Scroll Bar in the AHHASC System.**

Scroll bars are essential for navigating lengthy content, such as documents, web pages, and lists. By using the JScrollPane class to implement scroll bars in our system, we can view the contents that exceed the visible area of our screen. In this figure, as managers and technicians have to fill up numerous pieces of information that cannot be fitted into a single screen without scroll panes, our team has decided to use a scroll bar to maintain the simplicity of our GUI while accommodating more content.

Box Layout

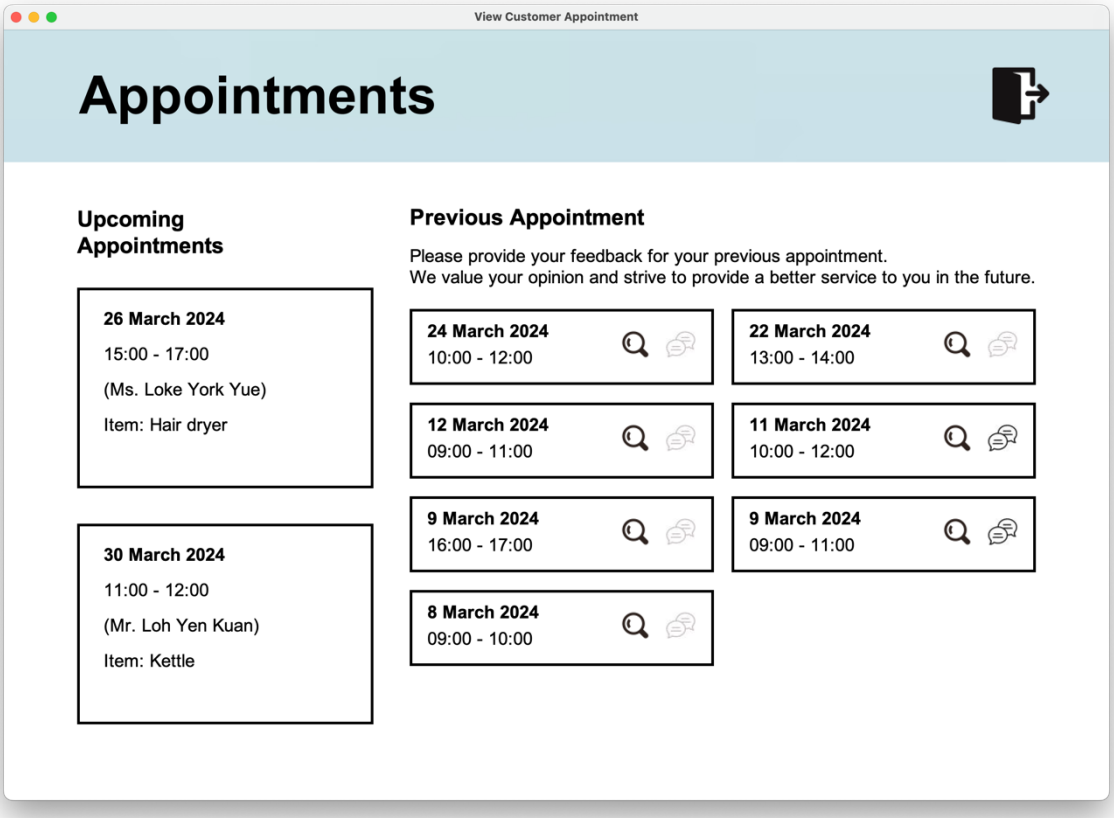

***Figure 44*: Demonstration of Box Layout on Customer Main Page.**

Our team has also utilized numerous layouts in our system's GUI. One of the most notable ones is the Box layout implemented in the Upcoming Appointments section for customers. The Box layout has simplified the process during coding, as we would just need to specify the preferred size of each "box" and its alignment before appending the information to each panel. Despite having its limitations, the Box layout is sufficient for us to systematically display the appointments to customers to improve their user experience with the system.

JTable

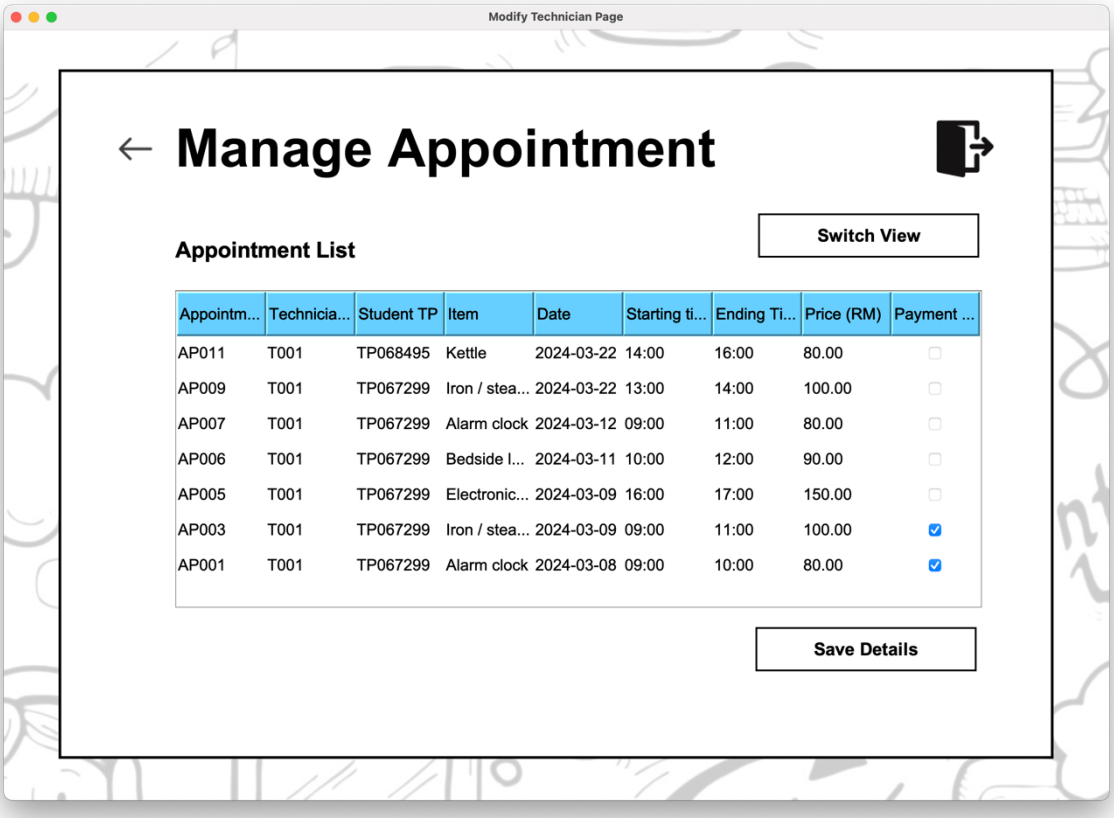

***Figure 45*: JTable Component on Modify Technician Page.**

To represent the data from the text file to users in a systematic way, our group has also implemented the JTable class to display all appointment details in a table form. Being a component that is easy to set up, having data represented in table form allows technicians to get a glimpse through the list of appointments quickly so that they can record the payment status of customers by just needing to click on the check box located on the right.

JDatePicker

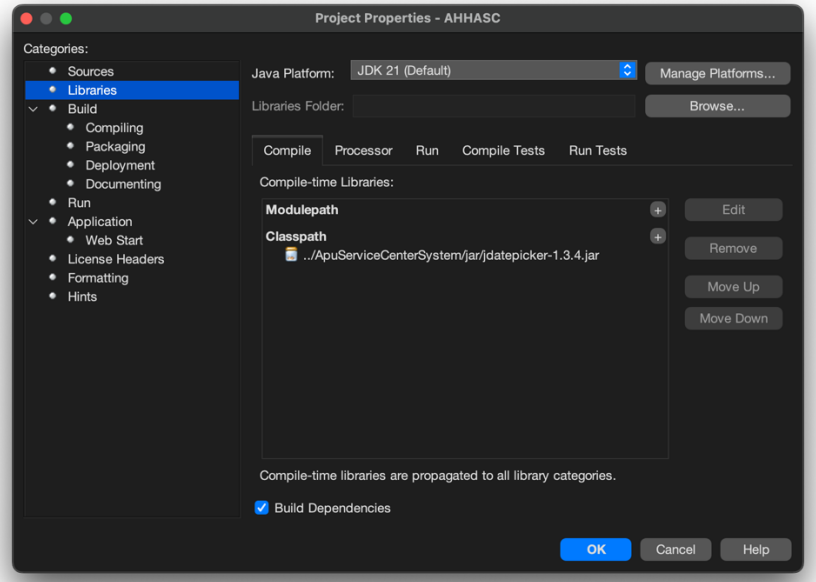

***Figure 46*: Project Configuration in Netbeans to Implement JDatePicker**

Since users would need to select a date for their appointments, our team has also implemented the JDatePicker library that provides the functionality of selecting dates via GUI. The library provides ready-to-use components that are easy to integrate and customize, allowing users to select dates conveniently using a calendar interface without needing us to worry about the text formatting of dates if users were to manually input the dates, which not only enhances the user friendliness of the application but also helps to reduce the formatting validation that has to be performed by the system.

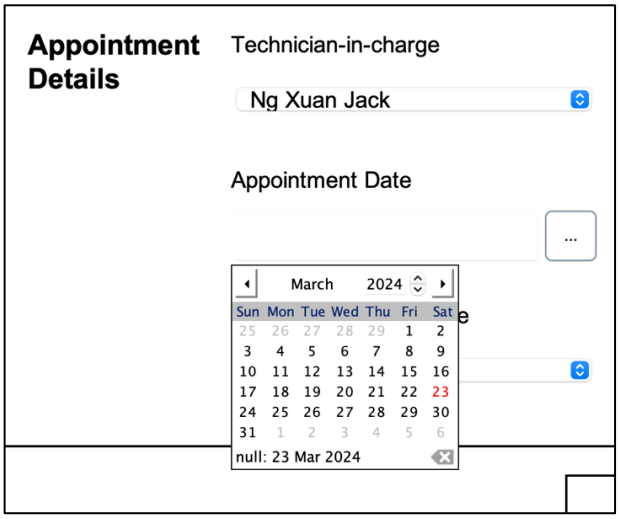

***Figure 47*: JDatePicker Implemented in the Create Appointment Page.**

Additional Java Concepts

Within our codes, we have also implemented a few extra concepts that provides more functionalities for our system. One of the most evident concepts is event listeners that allows our GUI to capture any events that have occurred and act accordingly (Wiener & Pinson, 2000). We will be explaining about the different types of additional event listeners, aside ActionListener, that we have implemented in our system.

```
public class BookAppointmentPage implements ComponentListener, MouseListener,
WindowListener, KeyListener {

    static JFrame frame;
    JLabel backgroundPicture, backArrow, logoutButton, promptTP, title, tpFront;
    JPanel backgroundPanel;
    JScrollPane fillInfoPane;
    JTextField tpField;
    JLayeredPane searchButton, cancelButton, saveButton;
    static Student currentStudent;
    InformationPaneComponent infoPaneComponent;

    public BookAppointmentPage() {

        frame = new JFrame("Appointment Booking Page");
        frame.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        frame.setLayout(null);
        frame.setSize(Asset.getFrameWidth(), Asset.getFrameHeight());
        frame.setLocation(Asset.getFramePositionX(), Asset.getFramePositionY());

        frame.addComponentListener(this);
        frame.addWindowListener(this);

        promptTP = new JLabel("Enter customer TP number:");
        promptTP.setFont(Asset.getNameFont("Plain"));
        promptTP.setSize(350, 50);

        tpField = new JTextField();
        tpField.setBorder(BorderFactory.createLineBorder(Color.BLACK, 2));
        tpField.setFont(Asset.getBodyFont("Plain"));
        tpField.setHorizontalAlignment(JTextField.CENTER);

        tpField.addKeyListener(this);

        searchButton = new Asset().generateButtonWithoutImage("Search", 150,
        promptTP.getHeight());
        searchButton.setFocusable(true);

        cancelButton = new Asset().generateButtonWithoutImage("Cancel",
        searchButton.getWidth(), searchButton.getHeight());

        cancelButton.addMouseListener(this);

    }
}
```

***Figure 48*. Constructor that Demonstrates the Implementation of Different Event Listeners.**

Component Listener

ComponentListerner enables handling component-level events, such as resizing, moving, or showing and hiding components, by offering methods that respond to these changes. In our code, we have added the ComponentListener to our frame, adjusting the size and positions of each GUI component automatically when the frame is resized.

```
@Override
public void componentResized(ComponentEvent e) {

    backgroundPicture.setBounds(0, 0, frame.getWidth(), frame.getHeight());

    backgroundPanel.setSize(frame.getWidth() * 9 / 10, frame.getHeight() * 9 / 10 - 30);
    backgroundPanel.setLocation((frame.getWidth() - backgroundPanel.getWidth()) / 2,
    (frame.getHeight() - backgroundPanel.getHeight()) / 2 - 15);

    backArrow.setLocation(backgroundPanel.getX(), backgroundPanel.getY() + 20);

    logoutButton.setLocation(backgroundPanel.getWidth() - logoutButton.getWidth() -
    50, backgroundPanel.getY() - 5);

    title.setBounds(backArrow.getX() + backArrow.getWidth() + 20, logoutButton.getY(),
    logoutButton.getX() - backArrow.getWidth() - backArrow.getX(),
    logoutButton.getHeight());

    promptTP.setLocation(title.getX(), title.getY() + title.getHeight() + 20);

    tpField.setBounds(promptTP.getX() + promptTP.getWidth(), promptTP.getY(), 200,
    promptTP.getHeight());

    tpFront.setLocation(tpField.getX() + 30, tpField.getY() + (tpField.getHeight() -
    tpFront.getHeight())/2);

    searchButton.setLocation(tpField.getX() + tpField.getWidth() + 20, tpField.getY()
    - 3);

    saveButton.setLocation(backgroundPanel.getWidth() - saveButton.getWidth() - 50,
    backgroundPanel.getHeight() - saveButton.getHeight() - 50);

    cancelButton.setLocation(saveButton.getX() - 20 - cancelButton.getWidth(),
    saveButton.getY());

}

@Override
public void componentMoved(ComponentEvent e) {
}

@Override
public void componentShown(ComponentEvent e) {
}

@Override
public void componentHidden(ComponentEvent e) {
}
```

***Figure 49*: Demonstration of the Implementation of ComponentListener.**

Mouse Listener

MouseListener handles mouse-related events, such as clicks, presses, releases, enters, and exits on a component. When these events occur, the program can execute customized actions based on user interactions with the mouse, such as changing mouse icons when the mouse hovers into the component or displaying a message when the element is clicked.

```
    @Override
    public void mouseClicked(MouseEvent e) {
    }
    @Override
    public void mousePressed(MouseEvent e) {
    }
    @Override
    public void mouseReleased(MouseEvent e) {
        if (e.getSource() == backArrow || e.getSource() == cancelButton) {
            int choice = JOptionPane.showConfirmDialog(frame, "<html>Do you wish to return
                         to the main page?<br>The inputted data will not be
                         saved.</html>", "System Exit", JOptionPane.YES_NO_OPTION,
                         JOptionPane.QUESTION_MESSAGE, new
                         ImageIcon(TextFileOperationsComponent.getPictureFilePath() +
                         "return_icon.png"));
            if (choice == 0) {
                Asset.setFramePosition(frame.getX(), frame.getY());
                ManagerMainPage.setFrameVisibility(true);
                frame.dispose();
            }
        } else if (e.getSource() == logoutButton) {
            int choice = JOptionPane.showConfirmDialog(frame, "<html>Are you sure that you
                         would like to logout from the system?<br>The inputted data will
                         not be saved.</html>", "System Exit", JOptionPane.YES_NO_OPTION,
                         JOptionPane.QUESTION_MESSAGE, new
                         ImageIcon(TextFileOperationsComponent.getPictureFilePath() +
                         "logout_icon.png"));
            if (choice == 0) {
                Asset.setFramePosition(frame.getX(), frame.getY());
                new LoginPage();
                frame.dispose();
            }
        }
    }
    @Override
    public void mouseEntered(MouseEvent e) {
        backArrow.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        logoutButton.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        cancelButton.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        saveButton.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    }
    @Override
    public void mouseExited(MouseEvent e) {
        backArrow.setCursor(Cursor.getPredefinedCursor(Cursor.DEFAULT_CURSOR));
        logoutButton.setCursor(Cursor.getPredefinedCursor(Cursor.DEFAULT_CURSOR));
        cancelButton.setCursor(Cursor.getPredefinedCursor(Cursor.DEFAULT_CURSOR));
        saveButton.setCursor(Cursor.getPredefinedCursor(Cursor.DEFAULT_CURSOR));
    }
```

***Figure 50*: Demonstration of the Implementation of MouseListener.**

Key Listener

KeyListener is a type of event listener that is triggered when we interact with our keyboard at a specific component. For our system, key listeners are usually implemented on text fields, where the system is coded to automatically trigger the corresponding button when the user pressed the Enter key from their keyboard.

```
@Override
public void keyTyped(KeyEvent e) {

}

@Override
public void keyPressed(KeyEvent e) {

}

@Override
public void keyReleased(KeyEvent e) {
    if (e.getSource() == tpField && e.getKeyCode() == KeyEvent.VK_ENTER) {
        MouseEvent clickEvent = new MouseEvent(searchButton,
                                MouseEvent.MOUSE_RELEASED,
                                System.currentTimeMillis(), 0,
                                searchButton.getWidth() / 2, searchButton.getHeight()
                                / 2, 1, false);
        searchButton.dispatchEvent(clickEvent);
    }
}
```

***Figure 51*: Demonstration of the Implementation of KeyListener.**

Window listener

WindowListener is triggered when an operation is carried out towards a frame or a window, such as the opening and closing of a window. For our system, we have included the *WindowListener* as we would like to validate that the user really wants to exit the system upon clicking on the close button to prevent any accidental closes that leads to information loss.

```
@Override
public void windowOpened(WindowEvent e) {
}

@Override
public void windowClosing(WindowEvent e) {
    if (e.getSource() == frame) {

        int choice = JOptionPane.showConfirmDialog(frame, "<html>Are you sure that you
                     would like to logout from the system?<br>The inputted data will
                     not be saved.</html>", "System Exit", JOptionPane.YES_NO_OPTION,
                     JOptionPane.QUESTION_MESSAGE, new
                     ImageIcon(TextFileOperationsComponent.getPictureFilePath() +
                     "logout_icon.png"));

        if (choice == 0) {
            Asset.setFramePosition(frame.getX(), frame.getY());
            new LoginPage();
            frame.dispose();
        }
    }
}

@Override
public void windowClosed(WindowEvent e) {
}

@Override
public void windowIconified(WindowEvent e) {
}

@Override
public void windowDeiconified(WindowEvent e) {
}

@Override
public void windowActivated(WindowEvent e) {
}

@Override
public void windowDeactivated(WindowEvent e) {
}
```

***Figure 52*: Demonstration of the Implementation of WindowsListener.**

Conclusion

To sum up, the AHHASC system is a hostel service system catered for three users: managers, technicians, and customers, each of whom plays a distinctive role in services related to repairing home appliances. The AHHASC system developed by our team aims to provide user-friendly interfaces with robust functionalities that allow the accommodation department of APU to focus on delivering a high quality service to the accommodation tenants. Alongside addressing the needs of managers, technicians, and customers, the AHHASC system also helps the personnel from the accommodation department to have optimized service booking operations that replace the traditional method of handling appointments. As our team developed and progressed into documenting our system, we looked into a few aspects, including flowcharts, user manuals, and object-oriented concepts that help develop and implement the system to provide value to its end users. Despite our team showcasing our thorough understanding of the requirements of the hostel management system by coding our system based on object-oriented concepts, there are still some limitations that we would wish to address. The usage of text files that might lead to security issues and the booking appointment process where customers would have to call the counter to book an appointment are some of the limitations that we have faced. Hence, it is hoped that future developers could work on providing more functionalities to the system, such as having an interface that allows customers to book the system by themselves, or integrating databases into the system so that all data can be handled swiftly and safely, to provide a more robust appointment management system that genuinely brings convenience to each of its end users.

References

Access Modifiers in Java – 2024. (8 November, 2023). Retrieved from Great Learning Team: https://www.mygreatlearning.com/blog/the-access-modifiers-in-java/#:~:text=Access%20modifiers%20are%20keywords%20that,protected%2C%20default%2C%20and%20private.

Bloch, J. (2018). *Effective Java: Third Edition*. Addison-Wesley.

Encapsulation - definition & overview. (2024). Retrieved from Sumo Logic: https://www.sumologic.com/glossary/encapsulation/#:~:text=Java%20collections%20encapsulate%20groups%20of,maintaining%20data%20integrity%20and%20security.&text=Classes%20in%20Java%20encapsulate%20properties%20and%20behaviors%20within%20a%20unit.

GFG. (2017). *Abstraction in Java*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/abstraction-in-java-2/

Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Stoughton, Massachusetts: Prentice Hall.

Sangai, S. (12 January, 2024). *Abstraction in Java*. Retrieved from Scaler Topics: https://www.scaler.com/topics/java/abstraction-in-java/

Schildt, H. (2014). *Java: A Beginner's Guide (Sixth Edition)*. McGraw-Hill Education.

Sierra, K., & Bates, B. (2005). *Head First Java*. Sebastopol: O'Reilly Media.

Vats, R. (2 October, 2022). *Modularity in Java Explained With Step by Step Example [2024]*. Retrieved from upGrad: https://www.upgrad.com/blog/modularity-in-java/

Wiener, R., & Pinson, L. J. (2000). *Fundementals of OOP and Data Structures in Java*. Cambridge, United Kingdom: Cambridge University.

Appendix

Workload Matrix

Tasks	Beh Swee Shen (TP068561)	Lim Beng Rhui (TP068495)
Documentation	80%	20%
Coding	20%	80%